Accelerating metabolic engineering through multimodal phenotype prediction with TorchCell and the Cell Graph Transformer (14/15 words)

Michael Volk[1,2,3*†], Aurosish Sharma[1,2,3†] and Huimin Zhao[1,2,3,4,5,6*]

[1]DOE Center for Advanced Bioenergy and Bioproducts Innovation, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

[2]Department of Chemical and Biomolecular Engineering, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

[3]Carl R. Woese Institute for Genomic Biology, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

[4]NSF iBioFoundry, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

[5]Center for Biophysics and Quantitative Biology, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

[6]NSF Molecule Maker Lab Institute, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

*Corresponding author(s). E-mail(s): mjvolk3@illinois.edu; zhao5@illinois.edu;

Contributing authors: ;

†These authors contributed equally to this work.

Abstract (≤150 words) [140/150]

Deep learning has advanced protein structure prediction, yet metabolic engineering lags because phenotype data are scattered across heterogeneous studies lacking a shared ontology. We present TorchCell, an open-source framework that annotates literature and public datasets with a shared ontology and ingests them into a Neo4j knowledge graph, then builds deep-learning-ready datasets in which each strain is a perturbation operator over a shared wildtype reference cell combining genome sequence, gene networks, metabolism, and environment. We develop the Cell Graph Transformer (CGT), a virtual-cell architecture in which biological interaction graphs constrain multi-head attention and an equivariant perturbation operator feeds multitask decoders. In *Saccharomyces cerevisiae*, CGT predicts trigenic gene-gene interactions (Pearson $\boldsymbol{r = 0.454}$), outperforming DANGO, DCell, and Yeast9 flux balance analysis, and generalizes to knockout expression ($\boldsymbol{r = 0.543}$) and morphology ($\boldsymbol{r = 0.619}$). CGT recommends gene deletions for beta-carotene and betaxanthin production, pairing with autonomous biofoundries for AI-guided strain engineering.

Keywords: metabolic engineering, foundation models, gene interactions, virtual cell, knowledge graph, strain design

Manuscript outline & status (editing view only) ✓ ready ■ author-reviewed, keep stable ✗ not done, agents may edit freely

Convention: each Results section = 3–4 paragraphs, high-level paragraph (P1) first; one main figure per section.

Main text

- ■ **Introduction**
- ✗ **Results** *(six sections, one per figure)*
  - ✗ **R1 – Shared ontology and knowledge graph unify data**
    - ■ Fig 1 – TorchCell + CGT overview (data a–c; model d–h)
    - ✗ P1 *(high level):* data fragmentation → one shared schema
    - ✗ P2: Biolink ontology, ingest-time validation, Neo4j KG, datasets-as-queries
    - ✗ P3: reference-cell + perturbation factorization; join/aggregation shrinks labels
    - ✗ P4: information budget (10^{10} vs 10^{8} bits); cut at H; baselines saturate fitness
  - ✗ **R2 – CGT predicts trigenic gene–gene interactions (state of the art)**
    - ✗ Fig 2 – gene–gene interactions + iBioFoundry design loop
    - ✗ P1 *(high level):* interactions are the hard structured label; CGT in one line
    - ✗ P2: trigenic interaction defined (peel off lower-order effects)
    - ✗ P3: results – SOTA vs Yeast9 FBA / DCell / DANGO (r=0.454)
    - ✗ P4: data/model scaling, graph-reg ablation, accuracy vs $|\tau|$
  - ✗ **R3 – One embedding, many phenotypes**
    - ✗ Fig 3 – expression, morphology, fitness
    - ✗ P1 *(high level):* one latent embedding generalizes across phenotypes
    - ✗ P2: expression prediction (r≈0.54) + diagnostics
    - ✗ P3: single-knockout morphology (r≈0.62)
    - ✗ P4: fitness + multitask sharing; why one embedding helps
  - ✗ **R4 – Natural genetic variation vs. model-designed perturbations**
    - ✗ Fig 4 – natural genetic variation
    - ✗ P1 *(high level):* natural variation vs designed perturbations
    - ✗ P2: natural strain variation across genome + environment
    - ✗ P3: model-designed (in-silico) gene perturbations
    - ✗ P4: comparison; what design adds beyond the natural range
  - ✗ **R5 – Drug exposure and representation robustness**
    - ✗ Fig 5 – drug exposure + robustness *(paper tag `r-robust`)*
    - ✗ P1 *(high level):* operator extends to drug/env; representation is robust
    - ✗ P2: drug-exposure prediction
    - ✗ P3: robustness – stable across splits; embedding + graph ablations
    - ✗ P4: graceful degradation as supervision shrinks
  - ✗ **R6 – Metabolism and strain design**
    - ✗ Fig 6 – metabolism + metabolic engineering *(paper tag `r5`)*
    - ✗ P1 *(high level):* ground the representation in metabolism; recommend targets
    - ✗ P2: metabolism integration (Yeast9 / gene–reaction)
    - ✗ P3: strain-design recommendations + iBioFoundry DBTL loop
    - ✗ P4: inference at scale
- ✗ **Discussion**
- ✗ **Methods** *(subsection status inline in `methods.tex`)*

Supplementary Information

- ■ **Note 1 – Functional equivalence of the perturbation operator**
  - ✗ Supplementary Fig. 1 – empirical equivalence (CGT- vs GNN-style, KO fitness)
  - ■ Setting · Concrete example · Origin of approximation · Precision · Relation to prior work · Empirical validation
- ■ **Note 2 – Graph-regularized attention contains graph attention**
  - ✗ Supplementary Fig. 2 – graph-reg relevance (λ sweep)
  - ■ Setting · Precision · Empirical validation

■ **Note 3 – Static-graph assumption and learning the perturbation**

■ Supplementary Table 1 – per-batch compute (two routes)
■ Static-graph assumption · Limitations · Single general data object · Compute & complexity · Empirical cost · Trade-offs

■ **Note 4 – Gene interactions as a learned function of fitness**

✗ Supplementary Fig. 3 – do interactions emerge from fitness?
■ Setting · Toward a formal statement · What would be surprising · Implication for metabolism · The atomic unit of data

✗ **Note 5 – Persistent entities and contingent observations** *(theory of the cut; Fig. 1c)*

✗ Supplementary Table 2 – persistent-entity corpora, measured from the public archives by `experiments/016-information-accounting`
✗ Setting · The measure (Prop.: compressed length bounds, not entropy) · The two corpora · Two asymmetries · Consequence · Objection · Precision
✗ *Open:* contingent-corpus gzip figure still computed on gilahyper – commit that script and regenerate Supplementary Table 6 with a compressed-size column

✗ **Note 6 – Classical-ML case study: representation & pooling** *(empirical counterpart of Note 5)*

✗ Supplementary Fig. 4 – classical-ML baselines (compose from `*_spearman_spearman_*_palette` bars + `node_embedding_performance_*_palette`) · Supplementary Table 3–5 (Spearman, MSE, Pearson)
■ Setup · Representation is identity · The barrier is pooling

■ **General SI figures** (follow the Notes)

■ Supplementary Fig. 6 – TorchCell overview schematic ■ Supplementary Fig. 7 – ontology data model (hourglass)
■ Supplementary Fig. 8 – KG normalization (convert/dedup/aggregate) ■ Supplementary Fig. 9 – DNA sequence selection
■ Supplementary Fig. 10 – DNA tokenization ✗ Supplementary Fig. 11 – expression & multitask diagnostics
✗ Supplementary Fig. 12 – inference at scale ■ Supplementary Fig. 13 – guilt-by-association annotation

■ **SI tables**: ■ Supplementary Table 1 compute ✗ Supplementary Table 2 entity corpora ✗ Supplementary Table 3–5 classical-ML ■ Supplementary Table 6 datasets ■ Supplementary Table 7 graphs

✓ready ■author-reviewed, keep stable ✗not done, agents may edit freely

Contents

1 Introduction (target 570 words) [362/570] ■

Mapping genotype to phenotype is a central goal of biology and the rate-limiting step of metabolic engineering, where the targets of interest – titre, rate, and yield – are polygenic phenotypes that emerge from many genes acting across transcription, translation, and metabolism [1, 2]. Unlike monogenic disorders, such phenotypes cannot be read from a single gene: their causal structure is obscured by epistasis and by the cellular layers that separate genotype from measurement [3, 4]. Growth is the most tractable of them – assays are cheap and scalable, gene–gene interactions are directly quantifiable, and the same readout spans applications from suppressing proliferation in disease to maximizing growth-coupled production.

Deep learning has transformed protein structure and function prediction, yet systems biology and metabolic engineering lag [5]. Two obstacles recur. First, phenotype data are scattered across small, heterogeneous studies with no shared schema, and broad genome-wide screens rarely interoperate with the deep, pathway-focused campaigns of metabolic engineering [6, 7]. Second, although multimodal measurements improve prediction, combining more than a few modalities is difficult and rarely scales [8]. Recent foundation cell models pretrain on single-cell gene-expression matrices and fine-tune for phenotype [9, 10], but industrial microbes lack such data; what they do have is diverse perturbation data and curated interaction graphs, including metabolism.

Existing approaches leave this gap open: constraint-based flux analysis captures metabolism but misses epistasis [2, 11], and recent perturbation transformers do not consistently beat linear baselines [12]. Our own classical baselines make the point sharply – with enough data they predict knockout fitness from almost any gene representation, yet fail on gene interactions (Supplementary Fig. 4) – so fitness alone does not justify a deep model, but the harder, structured labels do.

Here we present TorchCell, a schema-grounded knowledge graph that curates heterogeneous *S. cerevisiae* phenotype data into queryable multimodal datasets, in the data-standardization spirit of TorchGeo [13], together with the Cell Graph Transformer (CGT): a model that encodes a cell once, applies perturbations as a learned operator in embedding space, and folds biological networks into the training objective as a soft prior rather than a hard graph. It predicts fitness, gene interactions, gene expression, and morphology jointly, and needs only a genome – not an interaction graph – to run. We outline the data model and architecture in Supplementary Fig. 6.

2 Results (target 1,900 words) [679/1900] ✗

A shared ontology and knowledge graph unify metabolic-engineering data (380 words) [457/380] ✗**.** TorchCell confronts the data-fragmentation barrier by unifying heterogeneous phenotype data under a single schema and exposing it as deep-learning-ready datasets. Each experimental record is typed by an experiment ontology grounded in the Biolink Model and validated at ingestion – for acyclicity, subtype substitutability, Biolink conformance, and referential integrity – so that records from unrelated studies share one machine-checkable structure (Fig. 1a; ontology data model in Supplementary Fig. 7). Validated records are emitted as typed subgraphs into a Neo4j knowledge graph hosted at NCSA, and datasets are recovered as queries over this shared structure: a data-specific query returns one instance with its typed neighborhood, while a cross-experiment query joins studies through shared entities such as a common environment or genotype. This turns modular literature and public databases into interoperable, queryable datasets *[placeholder: TorchCell currently spans ∼N studies and ∼M labeled instances across fitness, gene interaction, expression, and morphology; dataset scale in Supplementary Table 6, integrated graphs in Supplementary Table 7].*

Rather than store a genome per strain, TorchCell factorizes data as a shared reference cell acted on by a perturbation (Fig. 1b). Each experiment is a tuple of a cell graph, an environment, a perturbation, and a phenotype; the minimal reference is a genome, over which optional gene, regulatory, protein-interaction, and metabolic networks add structure. The yeast knockout library, for instance, is a single reference with $\sim 10^{4-6}$ deletion strains, each represented as a perturbation over that reference rather than as an independently rebuilt graph. A mechanistic join-and-aggregation step then merges compatible records – recasting lethality to fitness, or pooling equivalent genotypes measured under different assays – enlarging datasets while shrinking the label vocabulary the model must decode (Supplementary Fig. 8).

This organization exposes a favorable information budget (Fig. 1c). The cell's molecular parts – DNA, RNA, protein, and small molecules – are *persistent entities*: a sequence is the same object in a glucose assay and in an oxidative-stress assay, and the public archives already hold $\sim 10^{13}$ compressed bits of them (Supplementary Table 2). Measured phenotypes are *contingent observations*, values pinned to one genotype, environment, and assay; the corpus integrated here amounts to $\sim 10^{10}$ compressed bits under the same measure. The two therefore differ by three orders of magnitude in scale and, more consequentially, in reuse: an entity representation serves every instance in which that entity appears, whereas a phenotype record constrains the map at one context and no other. We therefore cut the model at the shared representation, taking entity encoders already trained on the abundant side and spending the scarce labels only on the perturbation operator and the decoders (Supplementary Note 5). Consistent with this budget, classical baselines

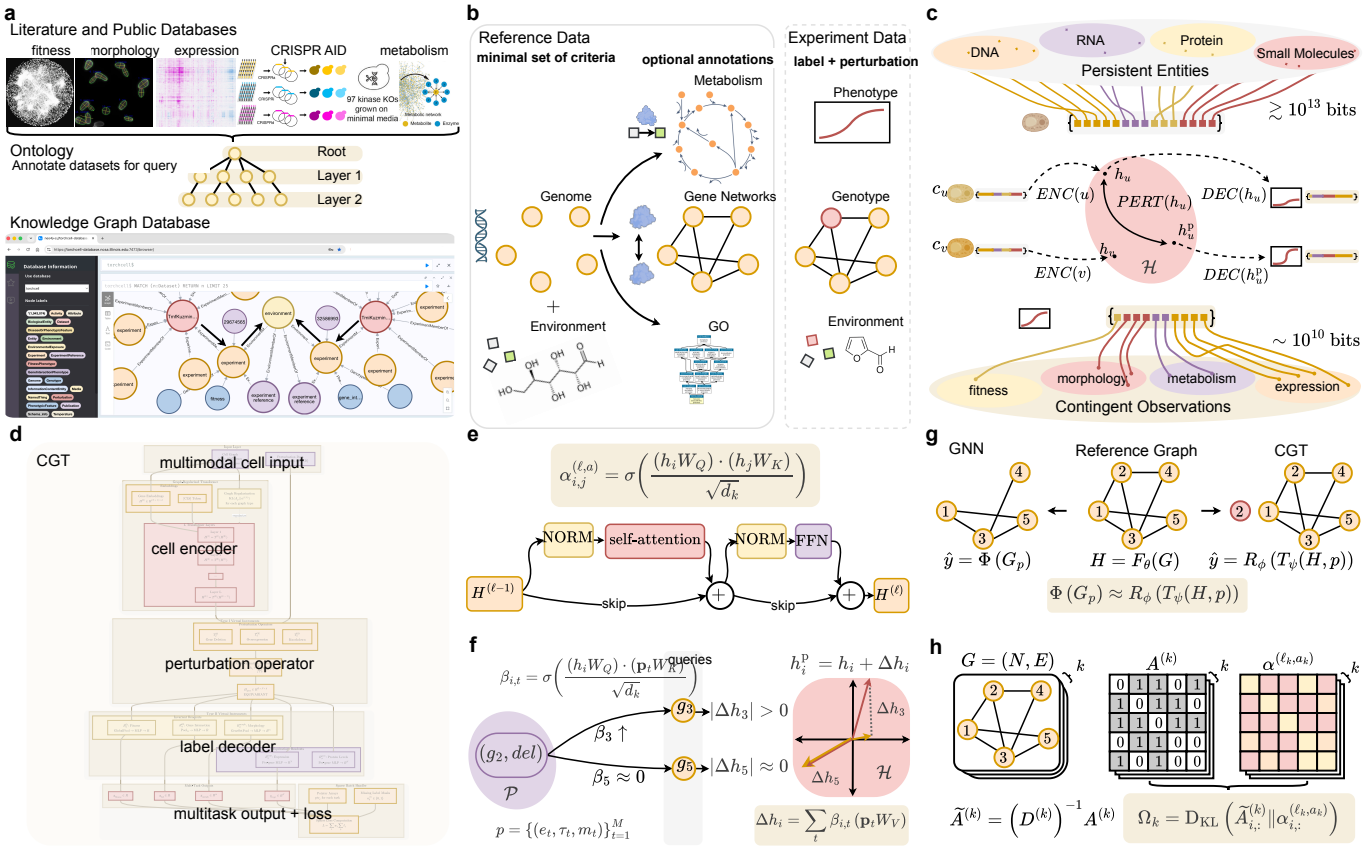


Fig. 1 TorchCell and the Cell Graph Transformer (CGT). **Top row – data.** (a) Literature and public databases are annotated with TorchCell's Biolink-grounded experiment ontology and ingested into a Neo4j knowledge graph (hosted at NCSA), queryable across datasets. (b) Each experiment is a shared reference cell (genome, gene networks) acted on by an experiment-specific perturbation (genotype, environment) with a measured phenotype; many strains reuse one reference. Gene, regulatory, and protein-interaction networks form a multigraph over genes; metabolism and Gene Ontology form bipartite graphs that can be mapped into the model or held aside as verification artifacts (for example, to test whether the model captures Gene Ontology it was not given; Methods). (c) Information accounting: molecular parts (DNA, RNA, protein, small molecules) enter as *persistent entities* – identities that do not change between assays, and whose public archives carry $\gtrsim 10^{13}$ compressed bits – and are mapped by encode (F_θ), perturb ($\mathcal{T}_\psi$), and decode ($\mathcal{R}_\phi$) into *contingent observations*, phenotypes measured in one genotype, environment, and assay, of which we integrate $\sim 10^{10}$ compressed bits. Bit counts are compressed lengths of the distributed archives, an upper bound on description length, not an information content. The model is cut at the shared representation H. Supplementary Note 5 gives the full accounting for this panel and states precisely what the bit counts do and do not mean; Supplementary Table 2 reports the measured size of every archive summed here, with its source and release. **Bottom row – model.** (d) CGT overview: multimodal cell input → cell encoder (ENC) → perturbation operator (PERT) → multitask decoder (DEC) → per-task outputs and loss. (e) Cell-encoder self-attention block: scaled dot-product attention $\alpha^{(\ell,a)}$ with pre-norm, feed-forward, and residual connections. (f) Perturbation operator: a soft cross-attention $\beta_{i,t}$ over perturbation tokens displaces each gene embedding, $h_i^{\mathrm{pert}} = h_i + \Delta h_i$; the same operator extends from gene deletions to drug and environment perturbations. (g) Functional equivalence: applying the operator to the reference representation approximates re-encoding the perturbed graph with a graph network, $\Phi(G_p) \approx \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$, so a strain is obtained without rebuilding or re-encoding its graph. The reference is encoded once and reused, and only the small-parameter operator runs per perturbation, amortizing compute across strains (proof and compute accounting in Methods and Supplementary Note 1). (h) Graph-regularized attention: each biological adjacency $A^{(k)}$ is row-normalized to $\tilde{A}^{(k)} = (D^{(k)})^{-1}A^{(k)}$ and matched to an attention head by a KL prior Ω_k that biases the head toward the biological graph: a soft constraint, not a hard mask that forces it, so supervision can overrule an edge the labels contradict (Methods and Supplementary Note 2).

predict knockout fitness from almost any representation yet fail on gene interactions (Supplementary Fig. 4) – so fitness alone does not justify a deep model, but the harder, structured labels do, motivating the architecture that follows.

The Cell Graph Transformer predicts trigenic gene-gene interactions at state of the art (380 words) [53/380] ✗. *[FILLER – R2.]* The Cell Graph Transformer (CGT; architecture in Fig. 1c and Methods) constrains attention with biological interaction graphs and maps the reference cell through an equivariant perturbation operator. On trigenic gene-gene interactions (GGI), where classical ML fails (Supplementary Fig. 4), CGT reaches Pearson $r = 0.454$ (Spearman 0.421), outperforming Yeast9 FBA, DCell, and DANGO (Fig. 2).

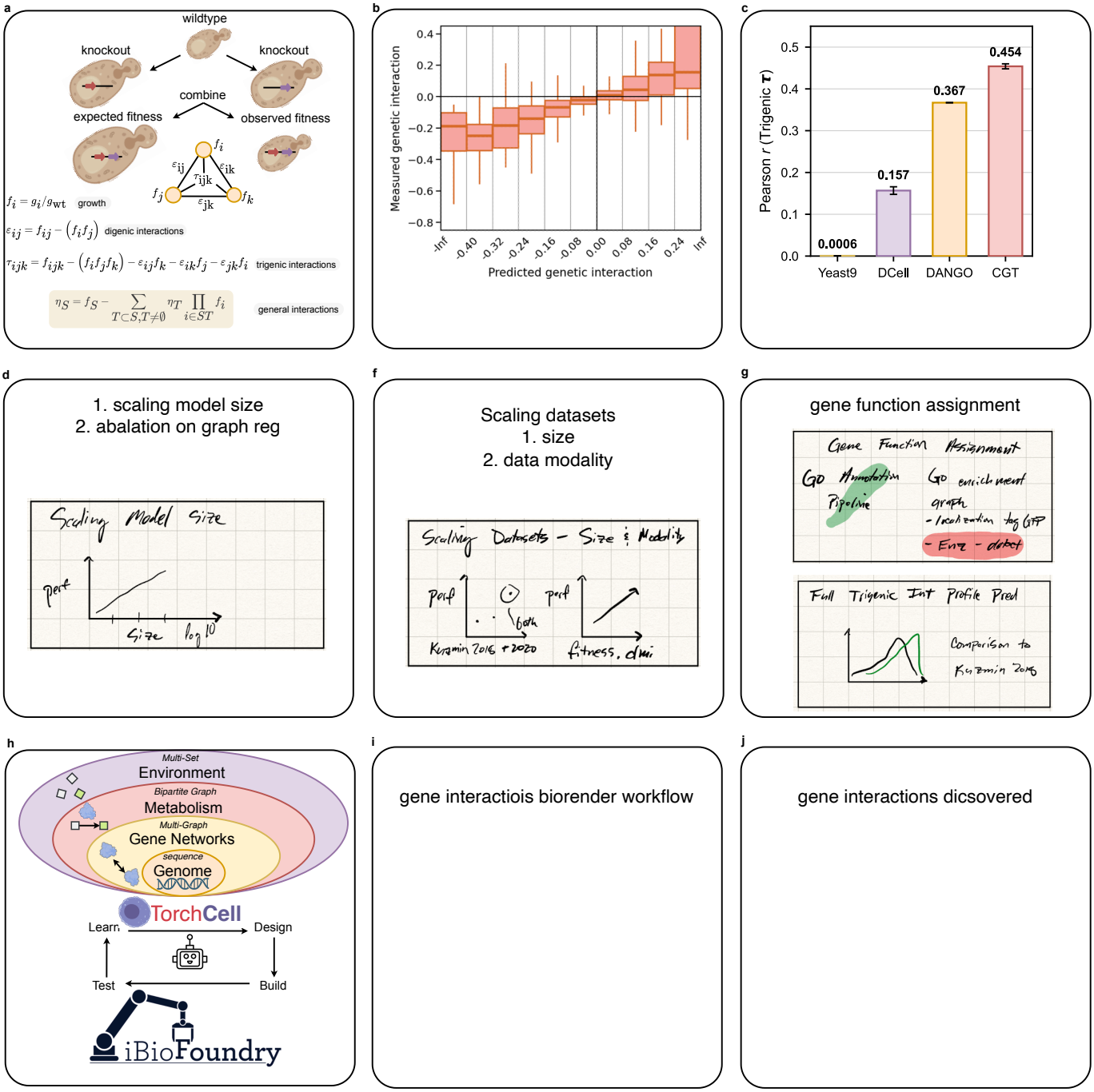


Fig. 2 TorchCell for gene–gene interaction prediction and strain design. (a) Higher-order genetic interactions are defined by subtracting lower-order effects: to isolate a k-way interaction you must peel off all interactions from subcollections of the same genes. For a trigenic interaction, the observed triple-mutant fitness is compared against the fitness expected from the single- and double-mutant terms, $\tau_{ijk} = f_{ijk} - f_i f_j f_k - \varepsilon_{ij} f_k - \varepsilon_{ik} f_j - \varepsilon_{jk} f_i$, with digenic interactions $\varepsilon_{ij} = f_{ij} - f_i f_j$. (b) The nested cell representation – genome, gene networks, metabolism, and environment – feeds TorchCell and a Learn–Design–Build–Test loop with the NSF iBioFoundry. (c) Predicted versus measured trigenic gene–gene interaction. (d) State of the art on trigenic interactions: Pearson r for Yeast9 FBA (0.0006), DCell (0.157), DANGO (0.367), and TorchCell (0.454).

One embedding, many phenotypes: expression, morphology, and fitness (380 words) [20/380] ✗. *[FILLER – R3.]* The same embedding predicts knockout expression ($r = 0.543$), single-knockout morphology ($r = 0.619$), and fitness (Fig. 3; expression diagnostics in Supplementary Fig. 11).

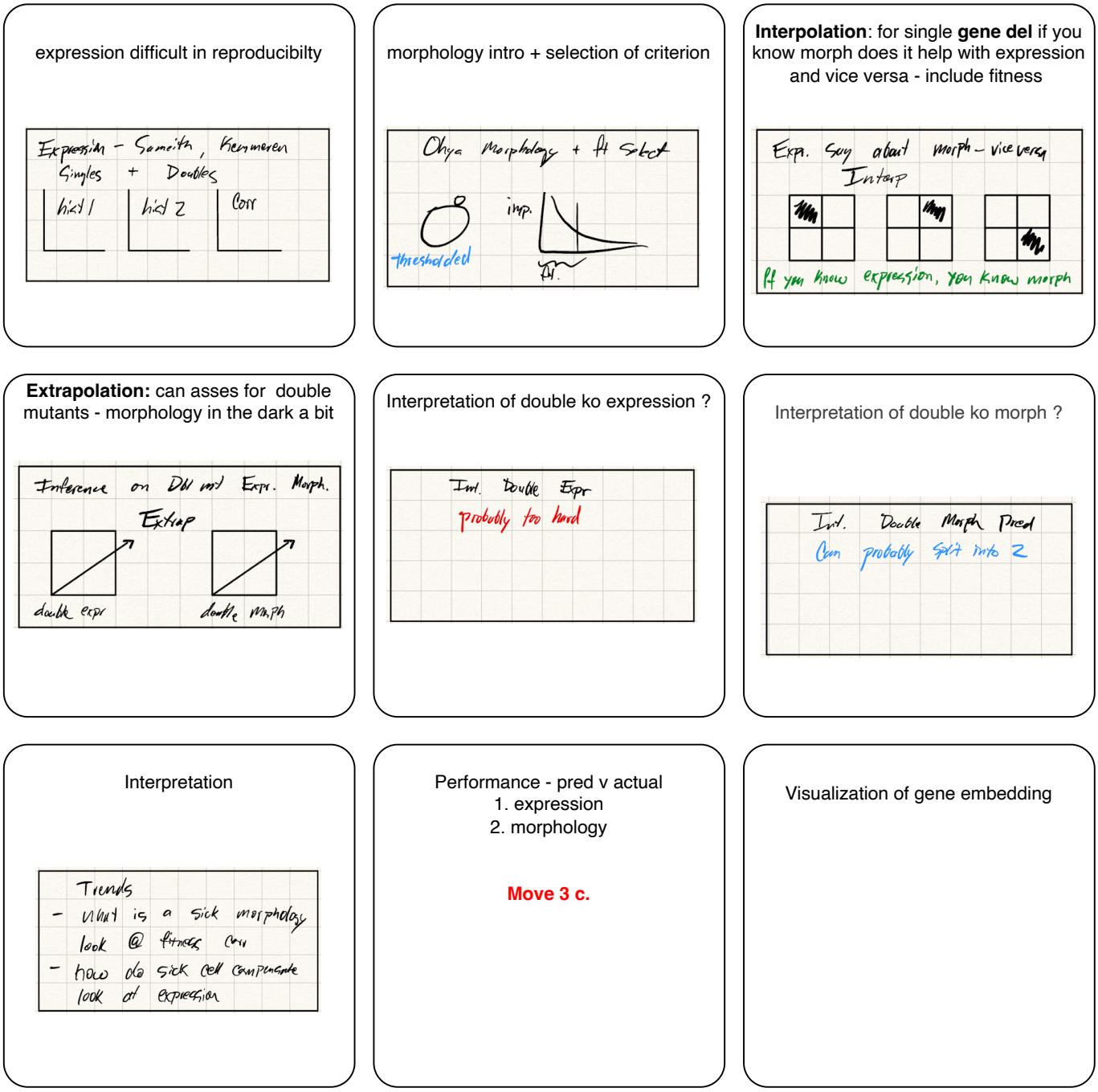


Fig. 3 One latent cell embedding generalizes across systems-biology phenotypes – expression, morphology, and fitness. (a)–(e).

Natural genetic variation versus model-designed perturbations (380 words) [21/380] ✗. *[FILLER – R4.]* We compare phenotypes from natural strain genetic variation against deep-learning-designed gene perturbations across genome and environment (Fig. 4).

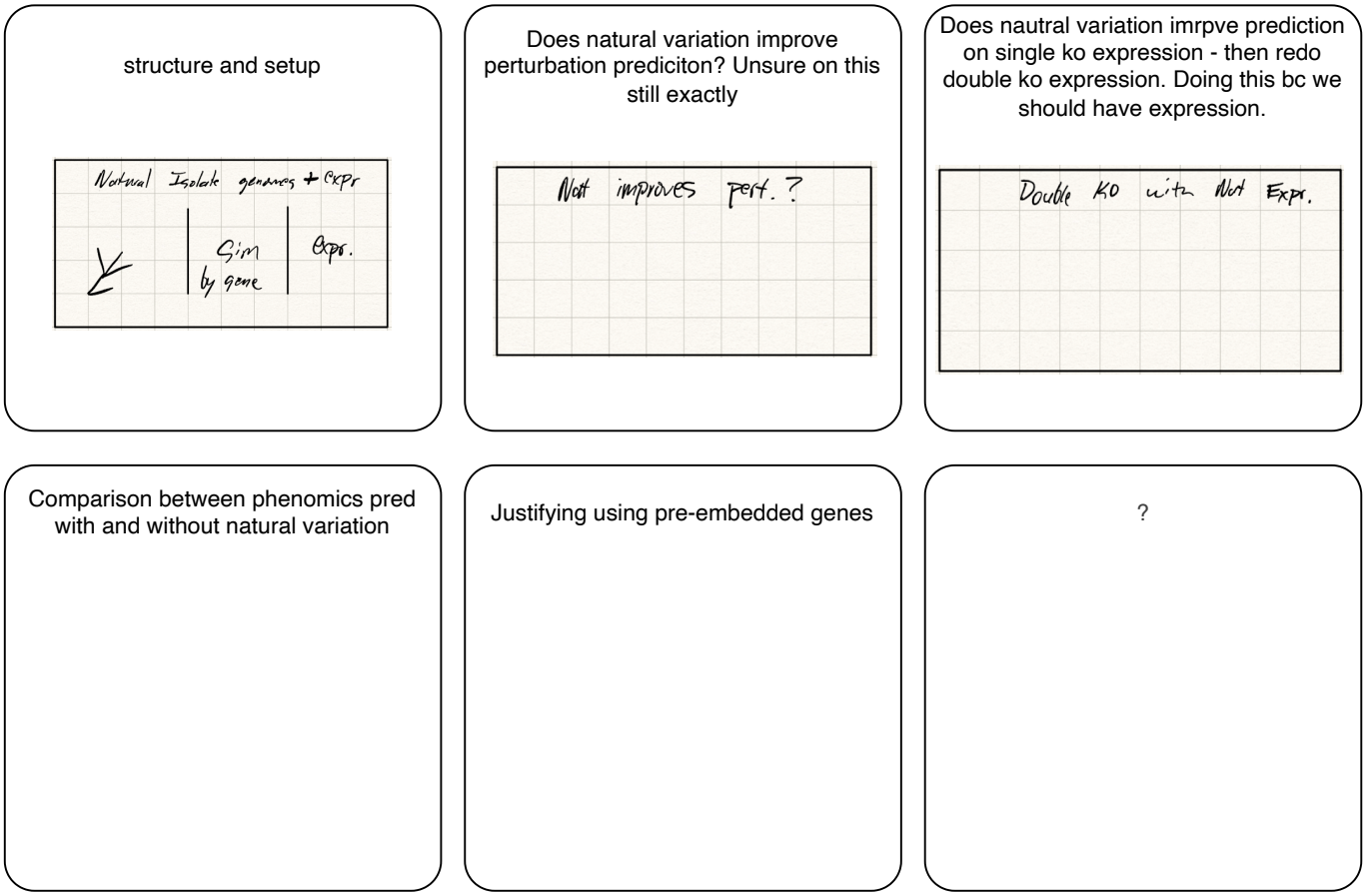


Fig. 4 Natural genetic variation versus model-designed perturbations. (a)–(c).

Drug exposure and robustness of the learned cell representation (380 words) ✗**.** *[FILLER – R-robust.]* The perturbation operator extends from gene deletions to drug and environment exposure, and the learned cell representation is robust: predictions stay stable across dataset splits, tolerate embedding and interaction-graph ablations, and degrade gracefully as supervision is reduced (Fig. 5).

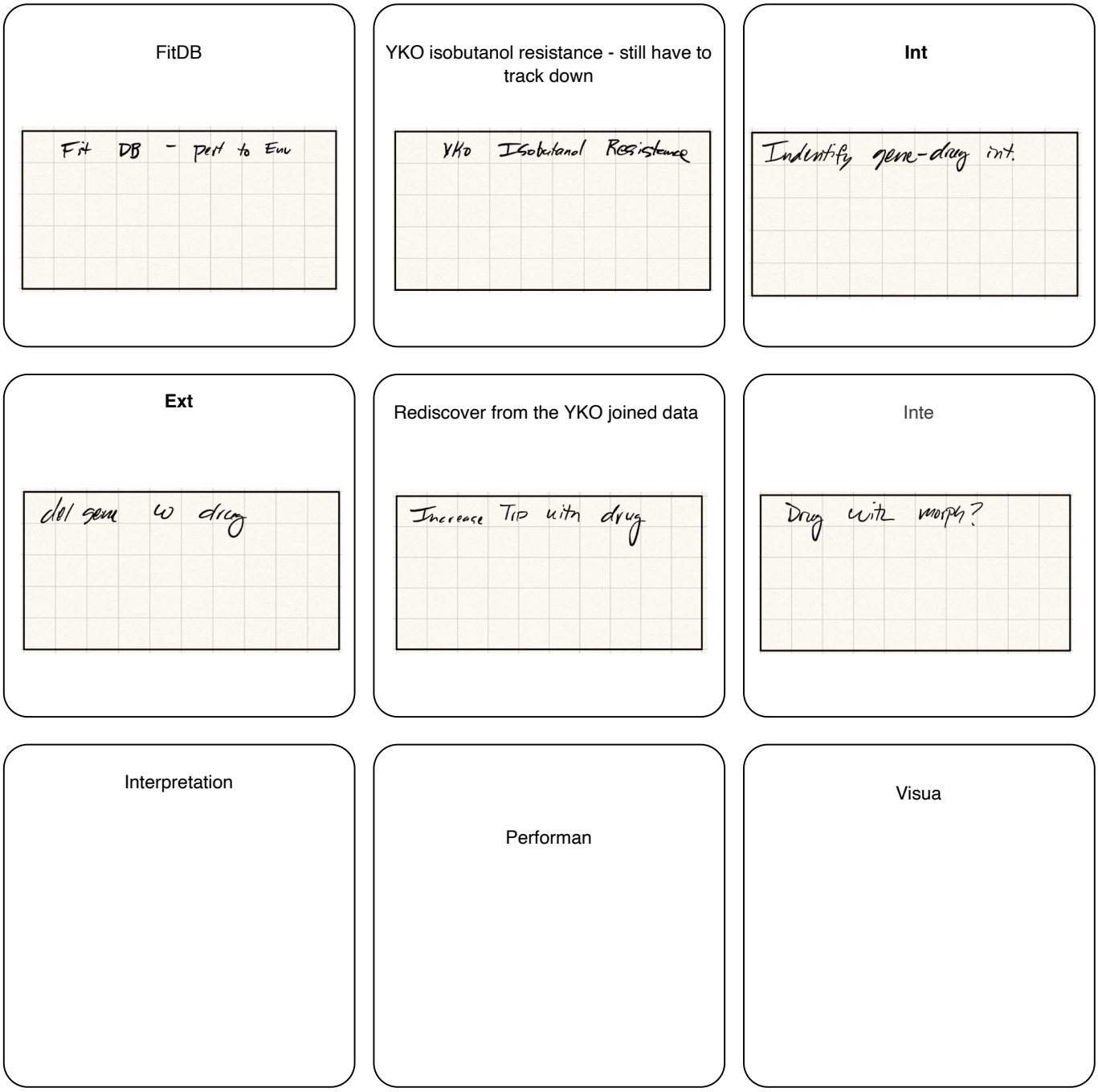


Fig. 5 Drug exposure and robustness of the learned cell representation. (a)–(f).

Extending to metabolism and strain design (380 words) [42/380] ✗. *[FILLER – R5.]* Grounding the cell representation in metabolism, CGT recommends gene targets for production phenotypes and pairs with the UIUC iBioFoundry for iterative design (Fig. 6; inference at scale in Supplementary Fig. 12).

Fig. 6 Extending TorchCell to metabolism and metabolic-engineering strain design. (a)–(f).

3 Discussion (target 570 words) [170/570] ✗

[FILLER – Discussion.] A strain-aware, biologically grounded representation is the differentiator; limitations include single-organism scope, morphology-data maturity, and error-bar provenance.

A central modeling assumption deserves explicit statement. TorchCell treats the catalogued biological networks (Fig. 1c) as annotations of a fixed reference and assumes the wiring is perturbation-invariant, so a strain is a thin operator applied to the reference rather than a rebuilt graph. This is a hypothesis, not a fact: an edge can effectively vanish when, for example, perturbing an upstream regulator drives a partner's expression so low that the interaction no longer occurs in the cell, and such condition-specific rewiring is not captured by a static adjacency. We adopt the assumption deliberately – it makes the perturbation a learnable displacement in representation space, keeps the biological graphs as a soft prior the data can overrule rather than a hard substrate, and yields a single general data object that different modeling paradigms can consume – and we treat its biological limits, computational advantages, and pipeline generality at length in Supplementary Note 3.

Table 1 Notation. The ENC/PERT/DEC stages (Fig. 1c) are implemented for CGT as $F_\theta/\mathcal{T}_\psi/\mathcal{R}_\phi$.

Symbol	Meaning
Sets and indices	
$\mathcal{U},\ \mathcal{I}$	encoded entities; labeled instances, $\mathcal{I} = \operatorname{supp} D$
$\mathcal{Y}$	phenotype space; y observed, $\hat{y}$ predicted
$\mathcal{P}$	perturbation space, $p \in \mathcal{P}$
N	number of nodes (genes); $N = 6607$, indexed $g_1, \dots, g_N$
$k,\ \ell,\ a,\ B$	relation type, layer, head, batch size
Graphs	
$G = (N, E)$	cell graph on N nodes with edge set E; relation-k adjacency $A^{(k)} \in \{0,1\}^{N\times N}$
$\tilde{A}^{(k)}$	row-normalized adjacency
Transformer	
H	cell representation $(h_{\mathrm{CLS}}, h_1, \dots, h_N)$, $h_i \in \mathbb{R}^d$
W_Q, W_K, W_V	query/key/value maps; head width $d_k = d/h$
$\alpha^{(\ell,a)}$	encoder self-attention map (rows sum to 1)
$\beta_{i,t}$	operator cross-attention weights
Constraint-based metabolism	
$S,\ v$	stoichiometric matrix $\in \mathbb{R}^{m\times r}$, flux; $Sv = 0$
ρ	gene–reaction association (GPR), gene $\mapsto 2^{[r]}$
Model	
Embed	entity encoder $\mathcal{U} \to \mathbb{R}^d$, or lookup $\mathbf{E}$
F_θ (ENC)	cell encoder, $H = F_\theta(G)$
p	perturbation set $\{(e_t, \tau_t, m_t)\}_{t=1}^M$: entity, type, magnitude; token $\mathbf{p}_t$, matrix $\mathbf{P} = (\mathbf{p}_1, \dots, \mathbf{p}_M)$ (keys/values)
ε	environment (media, temperature)
$\mathcal{T}_\psi$ (PERT)	perturbation operator, $H_{\mathrm{pert}} = \mathcal{T}_\psi(H, p)$
$\mathcal{R}_\phi$ (DEC)	decoder, $\hat{y} = \mathcal{R}_\phi(H_{\mathrm{pert}}, \varepsilon)$
$\Theta,\ \mathcal{L},\ D$	parameters (θ, ψ, ϕ), loss, data law over (G, ε, p, y)

4 Methods (no limit) [2472] ✗

Problem setup

We cast multimodal phenotype prediction as supervised learning of a single map from a cell to its phenotype. A model $\hat{f}_\theta$ reads a cell graph G, an environment ε, and a perturbation p and predicts a phenotype $\hat{y} = \hat{f}_\theta(G, \varepsilon, p)$, which we fit by minimizing the expected loss over the data distribution D,

$$\hat{\Theta} = \arg\min_{\Theta}\ \mathbb{E}_{(G,\varepsilon,p,y)\sim D}\big[\mathcal{L}\big(\hat{f}_\theta(G, \varepsilon, p),\ y\big)\big]. \tag{1}$$

The contribution is the structure of $\hat{f}_\theta$, which factors into encode, operate, and decode,

$$\hat{f}_\theta(G, \varepsilon, p) = \mathcal{R}_\phi\big(\mathcal{T}_\psi(F_\theta(G), p),\ \varepsilon\big), \tag{2}$$

so that abundant entity data trains the encoder F_θ while the scarce phenotype labels need only fit the perturbation operator $\mathcal{T}_\psi$ and the decoders $\mathcal{R}_\phi$. The notation, the data, and each factor are specified below.

Notation

We fix one notation, chosen to stay compatible with three literatures – machine learning on graphs, transformers, and constraint-based metabolic modeling – and to avoid the symbol collisions between them. Calligraphic letters denote spaces and learned operators. We cast the model as a three-stage framework – encode (ENC), perturb (PERT), decode (DEC), the stage names in Fig. 1c – instantiated for CGT by the encoder F_θ, perturbation operator $\mathcal{T}_\psi$, and decoder $\mathcal{R}_\phi$ (Table 1): ENC/PERT/DEC name the concept, $F_\theta/\mathcal{T}_\psi/\mathcal{R}_\phi$ the CGT implementation. Table 1 is the full glossary.

Ontology-grounded schema and knowledge graph

Every experimental record is typed by a schema Σ grounded in the Biolink Model: each class maps to a Biolink category and each relation to a Biolink predicate. At ingestion, records are validated programmatically against properties that keep the schema machine-checkable:

- *acyclicity* – Σ is a directed acyclic graph, so type and relation dependencies have no cycles and ingestion terminates;
- *Liskov substitutability* – each subtype is usable wherever its supertype is, so the class hierarchy is sound and a query on a supertype returns its subtypes;
- *Biolink conformance* – each class and relation resolves to a Biolink category/predicate, with field- and cardinality-level constraints;
- *referential integrity* – each typed edge connects declared node types.

Σ acts as an hourglass whose narrow waist forces heterogeneous sources into one typed structure. A raw record x is validated and emitted as a typed subgraph $\iota(x)$ into a Neo4j knowledge graph $\mathcal{K}$. Datasets are then queries over a shared structure: a *data-specific* query returns one instance with its nearest typed edges, while a *cross-experiment* query returns all instances sharing a structural entity (e.g. a common environment), joining studies through shared schema entities (Fig. 1a). This homogenizes modular literature and public data into queryable datasets, addressing the data-fragmentation barrier to deep learning in metabolic engineering (Supplementary Figs. 7, 8).

Datasets: a reference acted on by a perturbation

Each experiment is a tuple (G, ε, p, y) (Fig. 1b). The minimal *reference* is a genome – a sequence (FASTA) and an annotation (GFF) giving gene positions and sequence features; this alone is a valid record. Optional annotations add structure over that genome – gene, regulatory, and protein-interaction networks $\{A^{(k)}\}$, metabolism (the bipartite graph from (S, ρ)), and Gene Ontology – and are optional precisely because they presuppose the genome. Data factorize as *few references, many perturbations*: rather than store a genome per strain (the yeast knockout library is one reference and $\sim 10^{4-6}$ deletion strains), we store a shared reference G and a perturbation p over it, while natural isolates or divergent genomes become new references. An experiment additionally specifies a perturbation to the genome (p, the genotype) and possibly the environment ($\Delta\varepsilon$, as in a drug or tolerance study), together with a phenotype y. The phenotype may be a scalar such as fitness, a vector such as a gene-level expression or protein-abundance profile, or a tensor such as a set of morphology traits or metabolite concentrations, and the formulation is agnostic to its tensor rank. A mechanistic, explainable join/aggregation step merges instances across compatible conditions (e.g. across environments, or recasting lethality to fitness), enlarging datasets and shrinking the decoder's output domain $\mathcal{Y}$ to fewer symbol types (Supplementary Fig. 8).

Encoded entities

The cell's molecular parts – DNA, RNA, protein, and small molecules such as metabolites and drugs – form the universe $\mathcal{U}$ of *encoded entities*: each entity arrives with a representation $\mathrm{Embed} : \mathcal{U} \to \mathbb{R}^d$, supplied by a self-supervised model (a DNA, RNA, or protein language model, or a small-molecule encoder), a learnable lookup $\mathbf{E} \in \mathbb{R}^{N \times d}$, or a fixed featurization. Membership asks only that an entity *can be encoded*, not that it be pretrained. These encoders represent any candidate molecule rather than only those native to the organism, and they place similar entities near one another in $\mathbb{R}^d$, so the geometry is a usable prior: entities with similar representations are expected to act similarly on the cell.

The framing is deliberately *reductionist* – a cell is described by its parts – while an instance is *holistic*, a specific cell being a collection of entities assembled under a perturbation and an environment whose phenotype is a property of the whole. We hypothesize that the phenotype of a combination of entities can be estimated from supervised data, and that it takes far less phenotype data to do so because the entities arrive with knowledge already baked in: the self-supervised models supply statistical regularities of sequence and structure, and the genome annotation that defines each entity supplies curated structural knowledge. The encoder therefore begins from informative representations rather than from scratch, moving work off the scarce phenotype labels and onto data that is abundant. This factorization is a response to the limited-data regime; were phenotype data abundant, the simpler and more general choice would be to learn the cell-to-phenotype map end to end. Abundant entity data and scarce labels meet at the representation H, where we cut the model so that the labels fit only the operator and the decoder.

A measured information accounting motivates this cut (Fig. 1c; Supplementary Note 5). Both sides are measured with one instrument, the compressed length $L_C(D) = 8|C(s(D))|$ of a corpus D under a fixed serialization s and lossless compressor C. On the entity side we take the public archives in the representation in which each is actually distributed – nucleotide (NCBI `nt`), protein (UniProtKB/TrEMBL), small molecule (PubChem), and structure (wwPDB) – whose sizes are read from HTTP `Content-Length`, BLAST metadata, and the wwPDB index without downloading any

payload (Supplementary Table 2); they total $\sim 10^{13}$ bits. A counting bound corroborates the compressor: `nt` holds 4.06×10^{12} nucleotides, which a four-letter alphabet cannot write in under 8.1×10^{12} bits, and the archive is distributed in 7.1×10^{12}. On the label side, the phenotype corpus we integrate (Supplementary Table 6) occupies $\sim 1.4 \times 10^{9}$ compressed bytes, or $\sim 10^{10}$ bits – an asymmetry of $\sim 10^{3}$.

Scale is only half the argument, and the weaker half. An entity representation is a function of the entity alone, so one sequence informs the encoder for every instance in which that entity appears; a phenotype record is pinned to a single genotype, environment, and assay, and constrains the map nowhere else. Nor are these compressed sizes usable supervision: both are upper bounds on description length (Supplementary Note 5, Proposition 5), and the labels in particular are noisy and strongly redundant – interaction matrices are approximately low rank – which places the effective supervision nearer 10^{6}–10^{7} bits than 10^{10}. That discount only sharpens the conclusion. A sequence encoder carries 10^{8}–10^{10} parameters and the phenotype labels cannot pay for it, whereas the operator and decoder together carry $\sim 5 \times 10^{4}$ and they can. This is exactly the encode/operate/decode cut.

Multimodal cell representation

A cell is represented across nested biological scales (Fig. 1d). The genome is a sequence, the gene networks form a multi-graph with K relation types and adjacencies $\{A^{(k)}\}$, metabolism is the bipartite gene–reaction graph induced by (S, ρ), and the environment is a multi-set of conditions. Each gene g_i receives a node feature $x_i = \mathrm{Embed}(g_i) \in \mathbb{R}^d$ from its sequence and modality embeddings, and a learnable whole-cell token x_{CLS} is prepended to form the input states

$$H^{(0)} = (x_{\mathrm{CLS}}, x_1, \ldots, x_N) \in \mathbb{R}^{(N+1)\times d}. \tag{3}$$

The relation adjacencies $\{A^{(k)}\}$ are not consumed as message-passing edges. They enter the model only as priors on attention (Eq. 16), so the encoder reads the cell as a set of gene tokens whose interactions are learned rather than fixed in advance.

Cell Graph Transformer: forward pass

The encoder F_θ maps the input states $H^{(0)}$ to the cell representation $H = H^{(L)}$ through L transformer layers (Fig. 1e). Layer ℓ updates every token with multi-head self-attention followed by a position-wise feed-forward network. For head $a \in [h]$ of width $d_k = d/h$, the layer forms queries, keys, and values

$$Q^{(\ell,a)} = H^{(\ell-1)} W_Q^{(\ell,a)}, \qquad K^{(\ell,a)} = H^{(\ell-1)} W_K^{(\ell,a)}, \qquad V^{(\ell,a)} = H^{(\ell-1)} W_V^{(\ell,a)}, \tag{4}$$

and computes an attention map and head output

$$\alpha^{(\ell,a)} = \mathrm{softmax}\Big(\frac{Q^{(\ell,a)} \, (K^{(\ell,a)})^\top}{\sqrt{d_k}}\Big), \qquad O^{(\ell,a)} = \alpha^{(\ell,a)} V^{(\ell,a)}. \tag{5}$$

The heads are concatenated and mixed, then combined with the feed-forward network through residual connections,

$$\hat{H}^{(\ell)} = H^{(\ell-1)} + \big[\, O^{(\ell,1)} \,\|\, \cdots \,\|\, O^{(\ell,h)} \,\big] W_O^{(\ell)}, \qquad H^{(\ell)} = \hat{H}^{(\ell)} + \mathrm{FFN}^{(\ell)}\big(\mathrm{LN}(\hat{H}^{(\ell)})\big), \tag{6}$$

with layer normalization applied before each sublayer. During training, selected heads are aligned to biological graphs (Eq. 16), and this is the only place the networks $\{A^{(k)}\}$ act on the model. Because the wildtype cell graph G is shared across the many strains built on it, the encoder runs once per reference,

$$H = F_\theta(G) = (h_{\mathrm{CLS}}, h_1, \ldots, h_N), \tag{7}$$

and every perturbation reuses this H. The reparametrization replaces a per-strain re-encoding of perturbed graphs, of cost $\mathcal{O}(B\,L\,|E|)$, with one wildtype encode of cost $\mathcal{O}(B\,L)$ followed by the cheap operator below.

Perturbation operator

A strain is the reference acted on by a perturbation, and we realize this as a transformation within embedding space rather than a re-encoding. The operator $\mathcal{T}_\psi$ applies the perturbation set $p = \{(e_t, \tau_t, m_t)\}_{t=1}^{M}$ by cross-attention, in

which every cell token is a query and the perturbation tokens are the keys and values. Each perturbation token is embedded as

$$\mathbf{p}_t = r(e_t) + \mathbf{t}_{\tau_t} + m_t\,\mathbf{w}, \qquad r(e_t) = \begin{cases} h_j & \text{if } e_t = g_j \text{ is an in-cell gene,} \\ \mathrm{Embed}(e_t) & \text{otherwise,} \end{cases} \tag{8}$$

where $\mathbf{t}_\tau$ is a learned type embedding that carries the sign of the perturbation and $\mathbf{w}$ scales it by the magnitude. With $\mathbf{P} = (\mathbf{p}_1, \ldots, \mathbf{p}_M)$, every gene attends over the perturbation context and is updated by a weighted blend of the values,

$$\beta_{i,t} = \underset{t\in[M]}{\mathrm{softmax}} \frac{(h_i W_Q)\cdot(\mathbf{p}_t W_K)}{\sqrt{d_k}}, \qquad \Delta h_i = \sum_{t=1}^{M} \beta_{i,t}\,(\mathbf{p}_t W_V). \tag{9}$$

A residual block then produces the perturbed states,

$$\hat{h}_i = \mathrm{LN}(h_i + \Delta h_i), \qquad h_i^{\mathrm{pert}} = \mathrm{LN}\big(\hat{h}_i + \mathrm{FFN}(\hat{h}_i)\big), \qquad H_{\mathrm{pert}} = \mathcal{T}_\psi(H, p). \tag{10}$$

The update is a soft blend over the whole set, since $\sum_t \beta_{i,t} = 1$, and is not a dictionary lookup. Because p is a set, the operator is permutation-invariant in the perturbation tokens and accepts any number of them, and it is equivariant in the genes, so permuting genes permutes H_{pert} in step. The same operator therefore extends from gene deletions to drug, inhibitor, and environment perturbations by widening p, with no change to the architecture (Fig. 1f).

Two advantages follow from realizing a strain as an operator rather than a graph. First, no graph is constructed per strain: the wildtype is encoded once (Eq. 7) and each of the $\sim 10^{4-6}$ strains is obtained by applying $\mathcal{T}_\psi$ to the shared H, so the data pipeline never builds or re-encodes a perturbed graph, and the operator is a learned, functionally equivalent substitute for re-encoding – applying it to the reference representation approximates re-encoding the perturbed graph with a graph network, $\Phi(G_p) \approx \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$ (Fig. 1g; proved in Supplementary Note 1) – at $\mathcal{O}(BL)$ rather than $\mathcal{O}(BL|E|)$ cost. Second, the model does not require a graph at all: genes enter as a token set and interact through learned attention, with the networks $\{A^{(k)}\}$ acting only as an optional soft prior (Eq. 16), never as a message-passing substrate. Its minimal input is therefore a genome – a set of encodable entities – so interaction networks sharpen the model where available but are not a prerequisite, which lowers the minimal data criterion for a new organism or dataset.

Multitask readouts

Each phenotype is read out of the same perturbed representation by a task head $\mathcal{R}_\phi^t$, so one latent state serves every task and the heads differ only in how they pool over genes. Invariant heads predict a whole-cell scalar. The fitness head pools all genes,

$$\hat{y}_{\mathrm{fit}} = \mathrm{MLP}_{\mathrm{fit}}\Big(\tfrac{1}{N}\textstyle\sum_{i=1}^{N} h_i^{\mathrm{pert}}\Big), \tag{11}$$

and the genetic-interaction head pools the perturbed genes and compares them against the whole-cell token,

$$\hat{y}_{\mathrm{int}} = \mathrm{MLP}_{\mathrm{int}}\Big(\big[\, h_{\mathrm{CLS}}^{\mathrm{pert}} \,\big\|\, \tfrac{1}{|p|}\textstyle\sum_{g_j\in p} h_j^{\mathrm{pert}} \,\big]\Big). \tag{12}$$

Equivariant heads predict one value per gene, as for transcript or protein abundance,

$$\hat{y}_{\mathrm{expr},i} = \mathrm{MLP}_{\mathrm{expr}}\big(h_i^{\mathrm{pert}}\big), \tag{13}$$

and gene-set heads pool over a fixed set $\mathcal{G}_s$ of genes, as for the morphology traits of a cellular structure,

$$\hat{y}_{\mathrm{morph}} = \mathrm{MLP}_{\mathrm{morph}}\Big(\tfrac{1}{|\mathcal{G}_s|}\textstyle\sum_{i\in\mathcal{G}_s} h_i^{\mathrm{pert}}\Big). \tag{14}$$

The environment ε conditions every head, and together the encoder, operator, and heads realize the predictor $\hat{y}$ of Eq. (1). Adding a phenotype is adding a head, not changing the backbone.

Data splits and preprocessing

[FILLER.] Train/validation/test partitioning (and any gene- or interaction-disjoint splits); label normalization; handling of missing labels.

Graph-regularized attention and the training objective

A biological graph and an attention map are objects of the same shape (Fig. 1h). The adjacency $A^{(k)} \in \{0,1\}^{N\times N}$ over genes matches the gene–gene block of an attention map $\alpha^{(\ell,a)} \in [0,1]^{N\times N}$, so the two can be compared row by row. We row-normalize each adjacency into a distribution over neighbors,

$$\tilde{A}^{(k)} = (D^{(k)})^{-1} A^{(k)}, \qquad D^{(k)} = \operatorname{diag}\big(A^{(k)}\mathbf{1}\big), \tag{15}$$

and align a chosen head to graph k by penalizing the divergence of each row,

$$\Omega_k = \sum_{i\in\mathcal{I}_k} D_{\mathrm{KL}}\big(\tilde{A}^{(k)}_{i,:} \,\big\|\, \alpha^{(\ell_k,a_k)}_{i,:}\big), \tag{16}$$

where (ℓ_k, a_k) is the layer and head assigned to graph k and $\mathcal{I}_k$ is a sampled set of rows. The supervised term is a multitask loss over a batch of B instances. The tasks t index the phenotype heads and b indexes the instances in the batch. Most instances carry a label for only some tasks, so a mask $m_t^{(b)} \in \{0,1\}$, equal to 1 when the label $y_t^{(b)}$ is measured and 0 otherwise, restricts each term to the labels that exist,

$$\mathcal{L}_y = \sum_t w_t \sum_{b=1}^{B} m_t^{(b)}\, \ell_t\big(\hat{y}_t^{(b)}, y_t^{(b)}\big), \tag{17}$$

with task weights w_t. The full objective adds the graph priors to this supervised loss,

$$\mathcal{L} = \mathcal{L}_y + \sum_{k=1}^{K} \lambda_k\, \Omega_k. \tag{18}$$

Using the graphs as a soft prior rather than a hard graph-convolutional layer is deliberate. An edge with $A^{(k)}_{ij} = 1$ carries experimental evidence and is trusted, but $A^{(k)}_{ij} = 0$ usually means the pair has not been tested rather than that no interaction exists. A hard operator would commit to those uncertain zeros, whereas the KL prior only pulls attention toward the known edges and lets the data overrule it where the labels demand. Formally, this soft prior is the relaxation of masked graph-attention: a graph-attention layer confines each gene to its neighbors through an additive $-\infty$ mask on non-edges, and $\lambda\to\infty$ drives the regularized self-attention onto that same neighborhood, so the encoder is functionally equivalent to a graph-attention network while its forward pass stays mask-free. rather than masking is the right choice here because the graph is invariant across perturbations: a per-example mask re-imposes the same static adjacency on every forward pass and forces a rebuilt graph for each perturbed strain, whereas a constraint constant across examples is more naturally imposed once, in the objective, with the perturbation carried by the operator. Two further benefits follow. The same graphs can later be promoted from priors to prediction targets, and a head that keeps strong weight on a non-edge, where $A^{(k)}_{ij} = 0$ yet α_{ij} stays large, flags a candidate interaction that the networks have not yet recorded. Only gene–gene graphs are regularized. In the trigenic experiment these are the $K = 9$ physical, regulatory, and TFLink networks together with six STRING channels, one graph per head. Metabolism is bipartite and has no $N \times N$ adjacency, so it enters as a representation annotation and is never used as an attention prior.

Model training

[FILLER.] Optimizer, learning-rate schedule, batch size, early stopping; hardware (GPUs, distributed/DDP), wall-clock; software versions; random seeds.

Baselines

[FILLER.] DANGO, DCell, and Yeast9 flux balance analysis; classical-ML baselines (Supplementary Fig. 4); matched training data and evaluation protocol.

Evaluation metrics

[FILLER.] Pearson correlation (r) and any secondary metrics; aggregation across folds; definition of error bars (see Statistics).

Gene–gene interaction prediction

[FILLER.] Trigenic interaction prediction setup; performance vs. interaction magnitude; ablations (Fig. 2).

Expression and morphology prediction

[FILLER.] Knockout transcriptome and single-knockout morphology tasks; shared latent embedding; cross-phenotype transfer (Fig. 3).

CGT-guided strain design

[FILLER.] Objective and candidate enumeration for beta-carotene and betaxanthin production; ranking and selection of recommended gene deletions (Fig. 6).

Strain construction and cultivation

[FILLER.] Strains, plasmids, and genome edits; the TorchCell + UIUC iBioFoundry design–build–test–learn loop; cultivation and fermentation conditions.

Metabolite quantification

[FILLER.] Sampling, extraction, and analytical method (HPLC/LC–MS); calibration and titer quantification.

Inference at scale

[FILLER.] Large-scale inference pipeline and resources (Supplementary Fig. 12).

Statistics

[FILLER.] Statistical tests used and whether one- or two-sided; sample sizes (n); definition of center and dispersion; box-plot conventions (median, IQR hinges, whiskers $\leq 1.5\times$IQR); multiple-comparison handling; significance thresholds.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

Data availability

[FILLER.] Datasets are available on Hugging Face (https://huggingface.co/...); the Neo4j knowledge graph is hosted at NCSA (...); trained model weights are available at ..... Source data are provided with this paper.

Code availability

[FILLER.] TorchCell and the Cell Graph Transformer are open source at https://github.com/Mjvolk3/torchcell (license: ...), with a release archived at Zenodo (https://doi.org/...).

Acknowledgments.

Declarations

Supplementary Information ✗

Contents

Supplementary Note 1: functional equivalence of the perturbation operator ■

Encoding the wildtype cell once and applying a perturbation-conditioned operator is at least as expressive as rebuilding and re-encoding a separate graph for every strain. The operator can represent any predictor the rebuild-and-re-encode route can, and strictly more whenever the rebuilt graph would discard information the perturbation carries. Figure 1g and the Methods assert the approximation $\Phi(G_p) \approx \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$; this note separates it into two claims that are easy to conflate. The first is an exact reparameterization, a statement about which functions are expressible that does not depend on training. The second is a learned approximation: the trained operator matches the rebuild route only to the extent that the encoder loses no information and the operator has the capacity to fit the target.

Setting. There are two ways to predict a phenotype under a perturbation p. Perturbing the *data* materializes a rebuilt graph $G_p = \kappa(G, p)$, deleting or rewiring vertices and edges, and encodes it,

$$\hat{y} = \Phi(G_p).$$

The deterministic rebuild map $\kappa : (G, p) \mapsto G_p$ runs once per strain (written κ, not ρ, because ρ already denotes the gene–reaction association). Perturbing the *representation* encodes the reference once and operates on it,

$$\hat{y} = \mathcal{R}_\phi\big(\mathcal{T}_\psi(F_\theta(G), p), \varepsilon\big),$$

so the graph enters only through F_θ and the representation $H = F_\theta(G)$ is reused across all p. Collect the predictors each route can express into two function classes: $\mathcal{H}_{\text{data}} = \{\Phi \circ \kappa\}$, the maps built by perturbing the data, and $\mathcal{H}_{\text{rep}} = \{\Psi : \mathcal{G} \times \mathcal{P} \to \mathcal{Y}\}$, the maps expressible as a direct function of the pair (G, p), which is the amortized family CGT targets. Here Φ, Ψ are arbitrary measurable target maps and D is the data distribution over (G, p); remaining notation follows Table 1.

Proposition 1 (Reparameterization; amortization containment)**.**
With the function classes $\mathcal{H}_{\text{data}}$ and $\mathcal{H}_{\text{rep}}$ defined above, $\mathcal{H}_{\text{data}} \subseteq \mathcal{H}_{\text{rep}}$, with equality if and only if the rebuild map κ is injective on $\operatorname{supp} D$.

In words: the two routes target the same set of predictors, and the amortized route is strictly larger exactly when the rebuild destroys information. This is the amortization-containment property: the encode-once-and-operate family contains the data-rebuilding family, and strictly extends it whenever a feature or parametric perturbation such as dose or type survives in (G, p) but is erased by the rebuilt topology G_p.

Proof. We establish the two inclusions in turn.

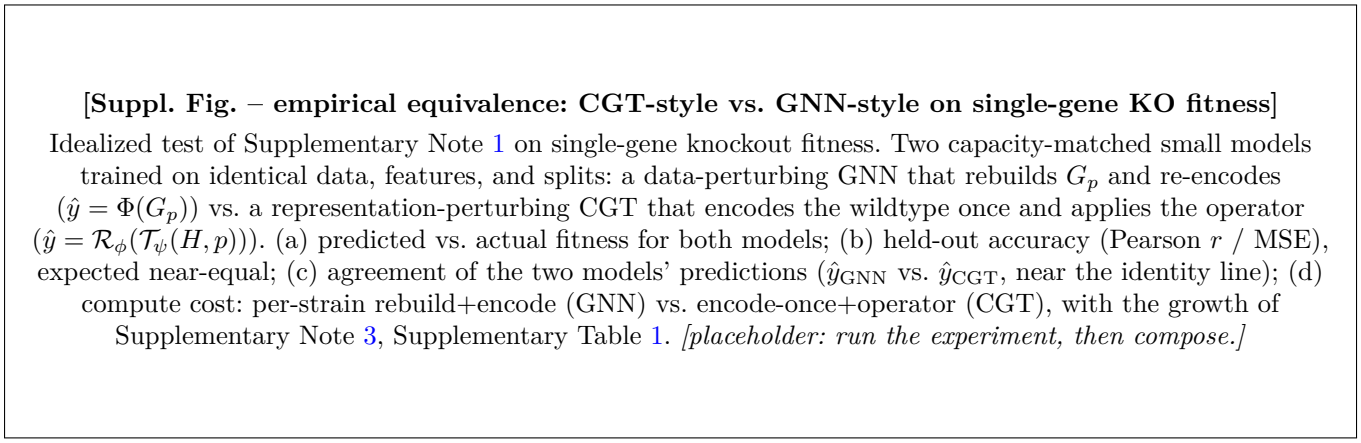
Containment. The map κ is deterministic, so (G, p) fixes G_p. For any target $\Phi^\star$ we have $\Phi^\star(G_p) = (\Phi^\star \circ \kappa)(G, p) =: \Psi^\star(G, p)$, an equality of functions, which gives $\mathcal{H}_{\text{data}} \subseteq \mathcal{H}_{\text{rep}}$.

Strictness. The reverse inclusion would require the pair (G, p) to be recoverable from the rebuilt graph alone, that is $\sigma(G_p) = \sigma(G, p)$ as generated σ-algebras. But the rebuild discards information: p is generally not recoverable from G_p, so $\sigma(G_p) \subsetneq \sigma(G, p)$ whenever κ is non-injective, and a map $\Psi \in \mathcal{H}_{\text{rep}} \setminus \mathcal{H}_{\text{data}}$ then exists. □

Concrete example. A three-gene example makes both the equality and its failure concrete. Take base graph G the path $A - B - C$. In the injective case, deleting B with $p = \{(g_B, \text{del})\}$ drops B and its edges, so $G_p = \{A, C\}$ with both nodes isolated, and the target $\Phi(G_p) = \#\text{edges} = 0$ is reproduced with no rebuild by $\Psi(G, p) = \#\text{edges}(G) - \#\{\text{edges incident to } B\} = 2 - 2 = 0$. In the non-injective case, overexpressing B at two doses, $p_1 = \{(g_B, \text{OE}, 1)\}$ against $p_2 = \{(g_B, \text{OE}, 3)\}$, changes no topology, so $\kappa(G, p_1) = \kappa(G, p_2) = G$ and any $\Phi(G_p)$ returns one value for both, yet the dose-response phenotypes differ. The pair (G, p) still carries the magnitude $m \in \{1, 3\}$ and separates the two cases; continuous dose, and likewise the perturbation type τ, lives in p and not in a bare rebuilt topology. This realizes $\mathcal{H}_{\text{rep}} \setminus \mathcal{H}_{\text{data}}$.

Origin of the approximation. Proposition 1 is about expressibility; it does not by itself say the trained CGT attains the match, and that is where the approximation sign originates. The ideal amortizer is $\Psi^\star = \Phi^\star \circ \kappa$. CGT realizes it not by that literal composition but by the learned $\mathcal{R}_\phi \circ \mathcal{T}_\psi$ acting on $H = F_\theta(G)$. Suppose F_θ is injective on $\operatorname{supp} D$, so that (H, p) carries the same information as (G, p), which is free for a fixed gene set because H keeps one identifiable row per gene; and suppose cross-attention over the perturbation set followed by an MLP readout is a universal approximator of set-conditioned permutation-equivariant maps. Then for every $\epsilon > 0$ there exist ψ, ϕ with

$$\sup_{(G,p)} \big\| \mathcal{R}_\phi\big(\mathcal{T}_\psi(F_\theta(G), p), \varepsilon\big) - \Psi^\star(G, p) \big\| < \epsilon, \tag{19}$$



Supplementary Figure 1 Empirical functional equivalence of the perturbation operator on single-gene knockout fitness (support for Supplementary Note 1).

and, with Proposition 1, $\mathcal{R}_\phi(\mathcal{T}_\psi(F_\theta(G), p), \varepsilon) \approx \Phi(\kappa(G, p)) = \Phi(G_p)$.

Precision. The two claims are distinct. The reparameterization of Proposition 1 is exact, a set-theoretic identity, because κ is deterministic. The trained $\Phi(G_p) \approx \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$ is a learned correspondence, not a mathematical identity: it holds to the extent that F_θ is lossless and $\mathcal{T}_\psi, \mathcal{R}_\phi$ can fit $\Phi^\star \circ \kappa$. The realizability conditions are standard: permutation-invariant set functions are universally approximable by the Deep Sets construction, self-attention is a universal approximator of permutation-equivariant sequence maps, and conditioning on a perturbation rather than refitting on modified data is amortization in the sense of hypernetworks. **[refs to add: Zaheer et al., *Deep Sets*, NeurIPS 2017; Yun et al., *Are Transformers Universal Approximators of Seq-to-Seq Functions?*, ICLR 2020; Ha, Dai & Le, *HyperNetworks*, ICLR 2017.]**

Relation to prior work. The closest precedent is the expressivity result that a structural graph transformation applied at preprocessing, followed by standard message passing, is at least as expressive as the corresponding improved message-passing algorithm; that result is topological and isomorphism-flavored, whereas Proposition 1 is a function-class containment for feature and parametric perturbations p (dose m, type τ) that a bare rebuilt topology cannot carry. Running a network on perturbed copies of a graph is known to exceed 1-WL expressiveness, by random node dropout or by aggregating a policy-drawn bag of subgraphs, with 1-WL the reference power for message passing. GEARS instantiates the data-graph side biologically, materializing perturbations as edges on a gene–gene graph [14]; CGT is its amortized counterpart, keeping the graph fixed and learning the perturbation in representation space. **[refs to add / search: Jogl, Thiessen & Gärtner, *Weisfeiler and Leman Return with Graph Transformations*, MLG 2022 (preprocessing transform ≥ improved MP); Papp et al., *DropGNN*, NeurIPS 2021 (node-dropout > 1-WL); Bevilacqua et al., *Equivariant Subgraph Aggregation Networks* (ESAN), ICLR 2022 (bag-of-subgraphs > 1-WL); Xu et al., *How Powerful Are GNNs?* (GIN), ICLR 2019 (1-WL bound).]**

Empirical validation. A minimal, idealized experiment on single-gene knockout fitness tests the equivalence directly (Supplementary Fig. 1). Two capacity-matched small models are trained on the same knockouts, gene features, and splits, differing only along the perturb-data versus perturb-representation axis: a data-perturbing GNN that rebuilds the perturbed graph $G_p = \kappa(G, p)$, deleting the knocked-out gene and its edges, and re-encodes, $\hat{y} = \Phi(G_p)$; and a representation-perturbing CGT that encodes the wildtype once, $H = F_\theta(G)$, and applies the operator, $\hat{y} = \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$. A single deletion gives an injective κ, so Proposition 1 predicts equal attainable accuracy: the amortized model should match the data-perturbing one on held-out fitness while encoding once rather than rebuilding and re-encoding per strain. The compute gap this avoids is quantified in Supplementary Note 3.

Supplementary Note 2: graph-regularized attention contains graph attention ■

A graph-attention network hard-wires each gene to attend only to its catalogued neighbors. CGT instead lets every gene attend to all genes and adds a soft penalty that pulls each attention row toward the graph. This note shows the two coincide in a precise sense: as the penalty weight λ grows, the soft head reproduces the hard-masked one exactly, and at any finite λ the soft head can do strictly more, either keeping attention on a pair the graph omits, which surfaces a candidate interaction, or pulling attention off a listed edge the data contradict, neither of which a hard mask permits.

Figure 1h and the Methods, in the graph-regularized attention subsection, state that penalizing the divergence of an attention head from a row-normalized adjacency is the soft relaxation of masked graph attention, and that $\lambda \to \infty$ makes the two coincide (Eqs. 15–18). This note makes that precise: hard-masked graph attention is the infinite-penalty limit of the KL-regularized head, and at finite λ the regularized head is strictly more expressive. Notation follows Table 1.

Setting. A graph-attention head confines gene i to its neighbors $\mathcal{N}(i) = \{j : A^{(k)}_{ij} = 1\}$ through an additive mask,

$$\alpha^{\mathrm{GAT}}_{i,:} = \mathrm{softmax}_j\big(s_{ij} + M_{ij}\big), \qquad M_{ij} = 0 \text{ on edges}, \quad M_{ij} = -\infty \text{ on non-edges},$$

so $\operatorname{supp} \alpha^{\mathrm{GAT}}_{i,:} = \mathcal{N}(i)$. The CGT head is unmasked,

$$\alpha^{\mathrm{CGT}}_{i,:} = \mathrm{softmax}_j\big(q_i \cdot k_j / \sqrt{d_k}\big),$$

and the graph enters only through the prior Ω_k (Eq. 16).

Proposition 2 (Graph attention as the $\lambda \to \infty$ limit)**.**
Fix a graph k and, for the head (ℓ_k, a_k) aligned to it, write $\lambda = \lambda_k$, $\tilde{A} = \tilde{A}^{(k)}$, and $\alpha_{i,:}$ for its attention row. Let $\mathcal{H}_{\mathrm{mask}}$ be the family of hard-masked graph-attention rows with support fixed to $\mathcal{N}(i)$, and $\mathcal{H}_{\mathrm{soft}}(\lambda)$ the family of KL-regularized rows under objective (18). Then $\mathcal{H}_{\mathrm{mask}} \subseteq \mathcal{H}_{\mathrm{soft}}(\lambda)$ for every λ, hard-masked graph attention is the $\lambda \to \infty$ limit of the regularized head, and at every finite λ the inclusion is strict.

Proof. The two parts match the two clauses of the proposition.

The limit. The supervised loss $\mathcal{L}_y$ is bounded in $\alpha_{i,:}$, being an average of bounded per-task losses, whereas the prior term $\lambda\, D_{\mathrm{KL}}(\tilde{A}_{i,:} \,\|\, \alpha_{i,:})$ scales with λ. As $\lambda \to \infty$ any minimizer of the full objective therefore drives $\alpha_{i,:}$ to the minimizer of $D_{\mathrm{KL}}(\tilde{A}_{i,:} \,\|\, \cdot)$. The forward KL is non-negative and, by Gibbs' inequality, equals 0 if and only if $\alpha_{i,:} = \tilde{A}_{i,:}$, and it is strictly convex in $\alpha_{i,:}$, so this minimizer is unique. Hence $\alpha_{i,:} \to \tilde{A}_{i,:}$, whose support is the graph neighborhood, $\operatorname{supp} \tilde{A}_{i,:} = \{j : A^{(k)}_{ij} = 1\} = \mathcal{N}(i) = \operatorname{supp} \alpha^{\mathrm{GAT}}_{i,:}$. The regularized head collapses onto the same neighbor-restricted attention that a hard $-\infty$ mask produces, so $\mathcal{H}_{\mathrm{mask}}$ is recovered in the limit and is contained in $\mathcal{H}_{\mathrm{soft}}(\lambda)$ for every λ.

Strictness at finite λ. Split the per-row penalty over the two kinds of column,

$$D_{\mathrm{KL}}(\tilde{A}_{i,:} \,\|\, \alpha_{i,:}) = \underbrace{\sum_{j:\, A^{(k)}_{ij}=1} \tilde{A}_{ij} \log \frac{\tilde{A}_{ij}}{\alpha_{ij}}}_{\text{edges}} + \underbrace{\sum_{j:\, A^{(k)}_{ij}=0} \tilde{A}_{ij} \log \frac{\tilde{A}_{ij}}{\alpha_{ij}}}_{\text{non-edges}}. \tag{20}$$

On every non-edge $\tilde{A}_{ij} = 0$ and $0 \log(0/\alpha_{ij}) = 0$, so the prior places no penalty on attention mass assigned off the graph, and this asymmetry is exactly what a hard mask lacks. A head may hold α_{ij} large where $A^{(k)}_{ij} = 0$ at zero cost to the prior, the only force on that mass being the supervised loss; reading the trained attention map and finding persistent off-graph mass, which is the signal that the model keeps populating an entry absent from the base graph, therefore flags a gene pair the model finds predictive that the catalogued networks have not recorded, a candidate missing interaction. On a true edge $\tilde{A}_{ij} > 0$, by contrast, driving $\alpha_{ij} \to 0$ sends its edge term to $+\infty$, so the prior resists dropping a catalogued edge, yet at finite λ a strong supervised gradient can still pull attention off an edge the labels contradict, trading a finite prior penalty for a better fit. A hard mask permits neither move: it sets the off-graph logit to $-\infty$ so that $\alpha_{ij} \equiv 0$ and the candidate edge is unrepresentable, and it makes the neighborhood structural and unoverridable. Any attention row with nonzero off-graph mass therefore lies in $\mathcal{H}_{\mathrm{soft}}(\lambda) \setminus \mathcal{H}_{\mathrm{mask}}$, giving $\mathcal{H}_{\mathrm{mask}} \subsetneq \mathcal{H}_{\mathrm{soft}}(\lambda)$ at every finite λ. □

Precision. Two caveats keep the caption honest. First, the equivalence is at the level of attention support, which genes attend to which, not of the within-neighborhood weights: the prior's target $\tilde{A}_{i,:}$ is uniform over $\mathcal{N}(i)$, whereas a graph-attention layer learns non-uniform neighbor weights, and at finite λ CGT's own $\alpha_{i,:}$ is data-shaped inside the softly enforced neighborhood. The defensible shared claim is same neighbors, learned weights, not identical weights.

[Suppl. Fig. – empirical relevance of graph-regularized attention: the λ sweep]

Idealized test of Supplementary Note 2. One cell-encoder head aligned to a single gene–gene graph, trained on held-out fitness with the graph-prior weight λ swept from 0 (free attention) to the hard-mask limit. (a) attention-to-adjacency divergence $D_{\mathrm{KL}}(\tilde{A}_{i,:} \,\|\, \alpha_{i,:})$ and support overlap with $\mathcal{N}(i)$ vs. λ, decreasing to 0 as $\lambda \to \infty$ (the masked graph-attention limit of Prop. 2); (b) held-out accuracy (Pearson r) vs. λ, peaked at intermediate λ, above both the free head ($\lambda = 0$) and the hard mask; (c) discovery: precision–recall of recovering held-out interactions from the top off-graph attention entries, against a hard-masked control that cannot place off-graph mass; (d) overruling: catalogued edges the head down-weights, enriched for label-contradicted pairs relative to a null. *[placeholder: run the λ sweep, then compose.]*

Supplementary Figure 2 Empirical relevance of graph-regularized attention: the finite-λ soft prior recovers graph attention in the limit, wins at intermediate λ, and enables edge discovery and overruling (support for Supplementary Note 2).

Second, as in Supplementary Note 1 the correspondence is learned, not a set-theoretic identity: it holds to the extent that the aligned head is trained toward $\tilde{A}$ and the supervised loss allows. The discovery-and-overrule asymmetry is a direct consequence of the KL direction $D_{\mathrm{KL}}(\tilde{A} \,\|\, \alpha)$, with the prior as reference and the attention as model; reversing the arguments would penalize off-graph mass and destroy discovery.

Proposition 2 is the companion of Proposition 1. Note 1 is the equivalence along the perturbation axis, operating on H rather than rebuilding G_p; this note is the equivalence along the graph-structure axis, regularizing attention rather than masking it. Together they justify the two architectural choices the bottom row of Fig. 1 depicts.

Empirical validation. A small, idealized experiment tests the proposition directly (Supplementary Fig. 2). A single cell-encoder head is aligned to one gene–gene graph and trained on held-out fitness while the prior weight λ is swept from 0, which is free attention, to large, which is the hard-mask limit. The proposition predicts three signatures. First, as λ grows the head's attention row $\alpha_{i,:}$ should converge to the row-normalized adjacency $\tilde{A}_{i,:}$, its support contracting onto the neighborhood $\mathcal{N}(i)$ and reproducing masked graph attention. Second, held-out accuracy should peak at an intermediate λ, above both the unconstrained head at $\lambda = 0$ and the hard-mask limit, locating the value in the soft regime. Third, at that intermediate λ the off-graph attention mass, meaning entries with $A^{(k)}_{ij} = 0$ yet α_{ij} large, should be enriched for gene pairs held out of the training graph but present in an independent interaction set, exhibiting the discovery channel, while catalogued edges the head down-weights should be enriched for pairs the labels contradict, exhibiting overruling. A hard-masked control produces neither signal by construction, which is the empirical face of the strict inclusion.

Supplementary Note 3: the static-graph assumption and the case for learning the perturbation ■

This note sets out the reasons for representing a strain as a perturbation of a fixed reference rather than as a rebuilt graph. They are of three kinds: a biological assumption together with its limits, a practical goal of one reusable data object, and a computational argument that the rebuild alternative does not scale.

The static-graph assumption. TorchCell treats the biological networks as catalogued objects that annotate a fixed reference (Fig. 1c, top): a genome is the minimal reference, and gene, regulatory, protein-interaction, and metabolic graphs are optional structure laid over it. A strain is then a perturbation applied to that shared reference rather than an independently rebuilt graph. This carries a biological hypothesis, that the catalogued wiring is perturbation-invariant: deleting or dosing a gene does not rewire the reference network, so the identified structures are essentially static across the strain library. The hypothesis is defensible for much of the graph, because a physical protein–protein interaction reflects a binding capability that a distant genetic perturbation usually leaves intact.

Limitations of the assumption. The assumption is a modeling choice, not a fact, and it has a clear failure mode. An edge that exists in the catalogue can effectively cease to exist in a particular strain: if perturbing an upstream regulator drives a partner's expression so low that the protein is essentially absent, the physical interaction cannot occur in that cell even though the reference graph still lists it. More generally, condition- and genotype-specific rewiring is not represented by a single static adjacency. Two features soften, though do not eliminate, this cost. First, the graph is a soft prior, not a hard substrate (Supplementary Note 2): the supervised loss can pull attention off an edge the labels contradict, and persistent off-graph attention can flag a context-specific interaction the catalogue omits. Second, the reference is the wildtype, so the assumption is only ever used to displace from a well-characterized baseline rather than to assert absolute structure. A genuinely dynamic, expression-gated edge is still only approximately handled, and we flag this as a limitation and a target for condition-conditioned graph priors.

A single general data object. Beyond expressivity (Supplementary Note 1), the design was chosen for a practical reason: keeping the reference fixed lets a data loader emit one general, read-ready object on request, which can then be consumed by different modeling paradigms without re-deriving the data. The same reference-plus-perturbation object is intended to serve, from one representation, machine learning on graphs, attention or transformer models, and constraint-based metabolic modeling; a stoichiometric or flux-balance formulation reads the metabolic subgraph, and a coupled-ODE kinetic model reads the same entities as state variables. Getting the data-transformation layer right once decouples the choice of modeling method from the data pipeline and makes multimodal method exploration a matter of consuming the same object differently. This was the original motivation, programmatic generality, and it is orthogonal to but reinforced by the expressivity and cost arguments below.

Compute and complexity. The alternative to learning the perturbation is to materialize and re-encode a perturbed graph per strain, which is expensive in a way that is easy to underestimate. Let N be the number of genes, $|E|$ the edges, d the hidden width, L the encoder depth, $|p|$ the perturbed genes per strain (with $|p| \ll N$), B the strains in a batch, and L_{op} the depth of the perturbation operator. Supplementary Table 1 tracks the growth of each route.

	Perturb data (rebuild)	Perturb representation (operator)
Encode reference	folded into per-strain	$\mathcal{O}(L\|E\|d)$, once
Per strain	$\mathcal{O}(L\|E\|d)$	$\mathcal{O}(\|p\|Nd)$
Batch of B strains	$\mathcal{O}(B\,L\|E\|d)$	$\mathcal{O}(L\|E\|d + B\|p\|Nd)$
Deeper operator, L_{op} layers over N genes	–	$\mathcal{O}(L\|E\|d + B\,L_{\text{op}}N^2d)$

Supplementary Table 1 Growth of per-batch compute for the two routes. The operator route wins by amortizing the encode across the batch and keeping the per-strain operator thin; a deep operator over all genes gives that advantage back.

The advantage has two sources, and both are necessary. The first is amortization: the encode $\mathcal{O}(L|E|d)$ is paid once and shared across the batch, so its per-strain share is $\mathcal{O}(L|E|d/B)$ and vanishes as B grows, whereas the rebuild route repeats the full encode for every strain and grows as $\mathcal{O}(B\,L|E|d)$. The second is thinness: the per-strain operator is a single cross-attention with $|p| \ll N$, costing $\mathcal{O}(|p|Nd)$, far below the rebuild's $\mathcal{O}(L|E|d)$. If the operator is deepened to L_{op} self-attention layers over all N genes, the per-strain cost climbs to $\mathcal{O}(L_{\text{op}}N^2d)$ and the batch term returns to $\mathcal{O}(B\,L_{\text{op}}N^2d)$, which is again processing a batch of B dense N-token graphs, the same fan-out the operator was meant to avoid. The design therefore keeps the operator shallow and adds capacity to fit the perturbation in the thin operator and the readout, not by re-encoding.

Empirical cost analysis. We tested this directly, and even in the simplest case, a single-gene deletion, optimizing the data step cannot close the gap. Building a strain by subgraphing the reference measured about 44 ms per sample

against about 0.4 ms for a thin perturbation-index representation, a gap that ranged from about 86 to 123× across configurations. An engineered zero-copy loader, which stores the full `edge_index` once, precomputes an $N \times K$ node-to-incident-edge index, and applies a per-strain boolean mask that zeros the edges touching perturbed genes in $\mathcal{O}(|p|\,d)$ rather than rebuilding in $\mathcal{O}(|E|)$, cut graph-processing time by 3.65× and edge-tensor memory by 93.7%, yet total per-sample time fell only 1.21× (from about 54 to 45 ms). Even precomputing a mask for every possible single deletion, the most favorable case available, reached only about 1.6× and never closed the residual 70-to-875× gap to the thin route. The reason is that the cost is not in loading, which profiled at essentially zero, but in the model: each strain in a batch carries a different mask, so message passing must be run separately on all B masked copies of the graph, inflating a 6,607-node forward pass to about 192,000 nodes and 2.4 million edge operations to about 67 million per batch. Masking rather than rebuilding only simulates the subgraph inside the same $\mathcal{O}(B\,L|E|)$ fan-out; it does not remove it *[numbers from the SubgraphRepresentation versus Perturbation and lazy-mask benchmarks; notes/experiments.006 subgraph-optimization and weekly 2025.44–45 reports]*. This is the empirical face of Supplementary Table 1: the encode cannot be avoided per strain on the rebuild route, no matter how fast the loader, which is exactly what the operator route amortizes.

Trade-offs between the two routes. The rebuild route is not strictly dominated. Because it encodes each strain directly, it never needs the wildtype embedding as an explicit object, whereas the operator route embeds the reference once and derives every strain from it, so every perturbation is referenced to the wildtype. In practice the operator route's single shared encode is the advantage, since the perturbations do originate from the wildtype and reuse dominates, but the trade-off is real, and these arguments, biological, computational, and pipeline-level, are early and warrant further work.

Supplementary Note 4: gene interactions as a learned function of fitness ■

We train one model to predict fitness and gene interactions together, and ask whether the interactions can be recovered from the fitness predictions alone, without the algebraic definition of an interaction ever entering the objective. If they can, it is evidence that a correctly trained model learns a complex nonlinear relationship from data rather than from a hand-written loss term. How far that generalizes is unclear, but the same logic would suggest that, with the right data, a model could come to respect the constraints of metabolism without an explicit physics- or flux-balance-informed regularizer. This note states the hypothesis, asks how much of it can be made a theorem, and marks the experiment as a placeholder.

Setting. For genes i, j, k let f_S denote the fitness of the mutant carrying the perturbations in $S \subseteq \{i, j, k\}$, normalized so the wildtype has fitness 1; a triple has seven such values, the singles f_i, f_j, f_k, the doubles f_{ij}, f_{ik}, f_{jk}, and the triple f_{ijk}. The trigenic interaction is a fixed multilinear map $g : \mathbb{R}^7 \to \mathbb{R}$ of these fitnesses,

$$\tau_{ijk} = g(f) = f_{ijk} - f_i f_j f_k - \varepsilon_{ij} f_k - \varepsilon_{ik} f_j - \varepsilon_{jk} f_i, \qquad \varepsilon_{ab} = f_{ab} - f_a f_b,$$

that is, what remains of the triple-mutant fitness after removing the multiplicative expectation and the pairwise interactions (Fig. 2a). Two ways to train follow. The *direct* route supervises τ_{ijk} as a label. The *factored* route supervises the fitnesses f_S and reads interactions off them, either by applying g to the predicted fitnesses or by a separate interaction head trained jointly; the test is whether the two agree even though g was never in the loss.

Toward a formal statement. Universal approximation is the wrong tool here: it guarantees that a large enough network can represent g composed with a fitness predictor, but it says nothing about whether training on fitness recovers τ, nor how accurately, so it is far too broad to be the claim. Two narrower statements are provable and sharpen the intuition.

Proposition 3 (Sufficiency: interactions need no extra capacity)**.**
If a representation H is sufficient for the sub-genotype fitnesses, meaning $f_S = \phi_S(H)$ for each S, then $\tau_{ijk} = g(\phi(H))$ is also a function of H. Any representation sufficient for fitness is therefore sufficient for interactions.

Proof. The claim is immediate. The composition of the measurable maps ϕ_S with the polynomial g is measurable, so τ_{ijk} is a deterministic function of H; predicting it requires no capacity, target, or objective term beyond those already needed to predict fitness. □

Proposition 3 is what makes the idea reasonable: the interaction is a fixed readout of quantities the model already predicts, so the definition g never has to appear in the loss. It does not, however, say the readout is easy to recover, because τ is a small residual of large terms.

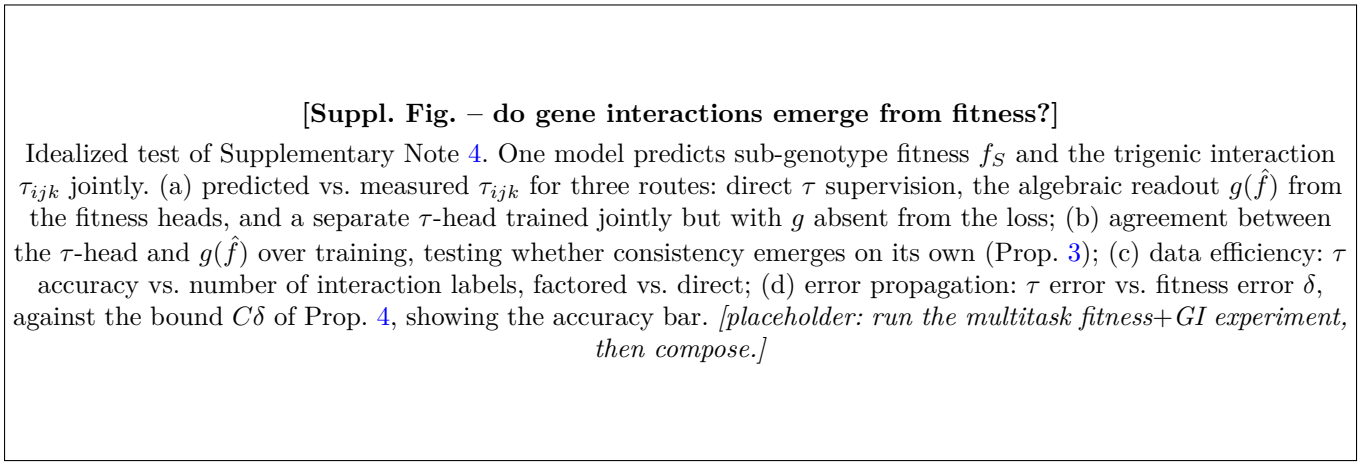
Proposition 4 (Conditioning: the accuracy bar)**.**
The map g is locally Lipschitz, and near the typical regime $f \approx \mathbf{1}$ its gradient is bounded by a constant $C = \mathcal{O}(1)$. A fitness estimate with $\|\hat{f} - f\|_\infty \le \delta$ then gives $|g(\hat{f}) - \tau_{ijk}| \le C\delta$. Since τ_{ijk} is a higher-order residual with $|\tau_{ijk}| \ll 1$, the relative error is $\approx C\delta/|\tau_{ijk}|$, which exceeds one unless $\delta \lesssim |\tau_{ijk}|/C$.

Proof. Each partial derivative of g is a sum of products of at most two fitnesses, bounded near $f \approx \mathbf{1}$; a first-order expansion gives the absolute bound, and dividing by $|\tau_{ijk}|$ gives the relative one. □

Together the two say the factored route is expressible for free but conditioning-limited: recovering interactions from fitness demands fitness accuracy finer than the interaction magnitude to resolve its sign and scale.

What would be surprising, and is only empirical. The claim we actually want to test is neither proposition. It is that joint training on the denser fitness signal makes an interaction head align with $g(\hat{f})$ *without* g in the loss, and generalizes better than direct τ supervision. This is a statement about inductive bias and data efficiency, not a theorem: whether optimization discovers the readout depends on the architecture, the data, and the training set-up. The favorable case is clear from Proposition 3, since a representation pinned down by abundant fitness labels is already sufficient for τ, but that it happens is for the experiment to show (Supplementary Fig. 3).

Implication for metabolism. If a measured phenotype is a fixed function of a latent cellular state the model predicts, the same reasoning suggests the model could come to satisfy the governing constraints without a physics-informed or genome-scale-metabolic regularizer in the loss: the constraint would be a readout, not an objective term. The caveat is identifiability. The trigenic case is clean because τ is a function of *observed* fitnesses, whereas metabolic constraints $Sv = 0$ involve fluxes v that are largely unobserved, so the constraint is not a function of measured phenotypes alone. Gene interactions are the fully observed instance where the idea can be tested before it is extrapolated to metabolism.



Supplementary Figure 3 Emergent recovery of gene interactions from fitness: whether a jointly trained model reproduces the interaction algebra without it appearing in the objective (support for Supplementary Note 4).

The atomic unit of data. The factored route trades per-instance flexibility, treating each strain as its own cell graph, for information density, since fitness labels are more abundant and lower-variance than interaction residuals, and it requires non-standard queries that assemble all sub-genotypes of a triple rather than one instance at a time. We pursue it because it may be the route that scales: rather than encode the interaction algebra, or the constraints of metabolism, in the objective, let the model learn them from data keyed to the atomic unit of the problem, the identity of the cell, its genome and strain. This is the bitter-lesson stance applied to phenotype prediction, and the reason the direction is worth the loss of flexibility.

Supplementary Note 5: persistent entities and contingent observations ✗

The molecular parts a cell is assembled from and the measurements made on cells are not two halves of one dataset. They differ in scale, by about three orders of magnitude, and they differ in kind. A protein sequence is the same object in a glucose assay and in an oxidative-stress assay; the measured phenotype is not. This note makes both statements precise, says exactly what the bit counts in Fig. 1c do and do not mean, and draws the consequence that determines the architecture: the model is cut at the shared representation H, entity encoders are taken already trained, and the phenotype corpus is spent only on the perturbation operator and the decoder.

Setting. Write $\mathcal{U}$ for the *persistent entities* – DNA, RNA and protein sequences, small molecules, and experimentally determined structures – and $\mathcal{I}$ for the *contingent observations*, the phenotype records of Supplementary Table 6. An entity $x \in \mathcal{U}$ is an identity, the same object in every experiment in which it occurs. An observation is a value measured on a cell in a context,

$$y \;=\; f(\{x_i\}_{i\in p},\; p,\; \varepsilon,\; a) \;+\; \eta, \tag{21}$$

with p the perturbation, ε the environment, a the assay, strain background and protocol, and η measurement noise. The entities in (21) recur in every observation that contains them. The observation itself is pinned to a single point of the context space and, being noisy, is not exactly reproducible even there. This asymmetry, rather than the raw size of the two corpora, is what forces the factorization.

The measure. Fig. 1c reports each corpus in bits. The quantity is the *compressed length*: for a corpus D, a fixed serialization s, and a fixed lossless compressor C,

$$L_C(D) \;=\; 8\,\big|C(s(D))\big| \quad \text{bits.} \tag{22}$$

This is a deliberately modest quantity. Because *information content* is used loosely and means two different things, Proposition 5 states which of them L_C is, and which it is not.

Proposition 5 (Compressed length is a bound, not an entropy)**.**
Let C be a lossless compressor with decompressor C^{-1} and self-delimiting output. For every corpus D,

$$K(D) \;\le\; L_C(D) \;+\; K\big(C^{-1}\big) \;+\; O(1), \tag{23}$$

where K denotes Kolmogorov complexity; and there is a sub-probability distribution q_C for which $L_C(D) = -\log_2 q_C(s(D))$. Thus L_C is at once a computable upper bound on the Kolmogorov complexity of D and an exact Shannon codelength under the model q_C that C implicitly assumes. It is not an estimate of the entropy of the source that generated D, and it depends on the choice of s and C.

Proof. Both halves follow from the definitions.

Kolmogorov bound. The compressed file together with the decompressor is a program that outputs D: run C^{-1} on $C(s(D))$ and deserialize. Its length is $L_C(D) + K(C^{-1}) + O(1)$, and $K(D)$ is by definition the length of the shortest program with that output, which gives (23). The bound is one-sided, and can be loose by any amount, because C removes only the redundancy it was designed to see.

Shannon codelength. C is injective with self-delimiting output, so its codewords satisfy the Kraft inequality, $\sum_D 2^{-L_C(D)} \le 1$, and $q_C(\cdot) := 2^{-L_C(\cdot)}$ is a sub-probability. Hence $L_C(D) = -\log_2 q_C(s(D))$ is the self-information of D under q_C. Nothing constrains q_C to resemble the true source, and the excess of L_C over the source entropy is exactly the Kullback–Leibler divergence between them. □

Two consequences govern how Fig. 1c should be read. Because L_C is an upper bound on each corpus and not an entropy, no single number in the figure is an information content. Because both corpora are measured with the *same* s and C, however, the comparison between them is like for like, and it is the ratio – to order of magnitude – that the figure claims.

The persistent-entity corpora. Supplementary Table 2 evaluates (22) for the public archives in the representation in which each is actually distributed. No payload is downloaded: the sizes come from HTTP `Content-Length`, from NCBI BLAST metadata, and from the wwPDB directory index, so every entry is re-derivable by a reader with a network connection. Nucleotide sequence dominates, and the total is

$$L_C(\mathcal{U}) \;\approx\; 9.8\times10^{12} \;\approx\; 10^{13} \text{ bits}, \tag{24}$$

falling only to 9.2×10^{12} if RefSeq transcripts are dropped as already contained in `nt`. An independent count corroborates the compressor. NCBI `nt` holds 4.15×10^{12} nucleotides, which a four-letter alphabet cannot write in fewer than 8.3×10^{12}

Supplementary Table 2 Persistent-entity corpora. Compressed size of each public archive in the representation it is actually distributed in, obtained without downloading any payload (HTTP `Content-Length`, NCBI BLAST metadata, or the wwPDB directory index). Compressed size $L_C(D) = 8|C(s(D))|$ is a computable upper bound on Kolmogorov complexity and a codelength under `gzip`'s implicit model, not an intrinsic information content; it is applied identically to both worlds of Fig. 1c, so the *ratio* is what is claimed. RefSeq transcripts also occur in `nt`, so the conservative total omits them. Sizes are decimal GB (10^9 bytes); PubChem and the PDB are rolling archives, so their *Release* is the fetch date. Measured 2026-07-13 (04:02:43 UTC) by the script named above and archived as a dated snapshot; no number here is hand-entered. These archives grow monotonically, so re-running the script returns a *larger* entity total and a larger ratio – the separation reported here is conservative.

Modality	Corpus	Release	Contents	GB	Bits
Protein	UniProtKB/TrEMBL	2026_02	149,234,636 sequences	40.5	3.2×10^{11}
Protein	UniProtKB/Swiss-Prot	2026_02	575,503 sequences	0.1	7.5×10^{8}
DNA / nucleotide	NCBI nt	2026-07-08	128,624,482 sequences (4.1×10^{12} nt)	884.1	7.1×10^{12}
RNA (transcripts)	NCBI RefSeq RNA	2026-06-26	81,647,541 sequences (2.4×10^{11} nt)	67.4	5.4×10^{11}
Small molecule	PubChem Compound	2026-07-13	357 SDF blocks, CID 1–178,500,000	115.8	9.3×10^{11}
Structure	wwPDB mmCIF	2026-07-13	255,710 structures	89.2	7.1×10^{11}
Total (all rows)				1,197.1	9.6×10^{12}
Total (non-overlapping: `nt` + TrEMBL + PubChem + PDB)				1,129.6	9.0×10^{12}

bits, and the archive is distributed in 7.2×10^{12}: the compressed size recovers 87% of a counting bound derived without reference to any compressor. The two routes agree, so (24) measures sequence content rather than an artifact of file formatting.

The contingent-observation corpus. The phenotype corpus integrated here (Supplementary Table 6) holds $\sim 10^7$–10^8 labeled instances across the studies it unifies. Serialized as the per-study label tables and compressed with the same instrument, it occupies $\sim 1.4 \times 10^9$ bytes, so

$$L_C(\mathcal{I}) \;\approx\; 1.1 \times 10^{10} \;\approx\; 10^{10} \text{ bits}, \tag{25}$$

about three orders of magnitude below (24). The corpus aggregates the large-scale yeast genotype–phenotype studies available to us under a single ontology, which is the point: the separation in Fig. 1c is not an artifact of having assembled a small phenotype corpus, and it would survive the addition of any comparable study.

Two asymmetries. The first is scale, and it is the weaker of the two: $L_C(\mathcal{U})/L_C(\mathcal{I}) \approx 10^3$. The second is persistence, and it is the reason the first matters. An entity representation is a function of the entity alone, so one sequence informs the encoder for *every* instance in which that entity appears, across assays, environments and laboratories. A phenotype record, by (21), constrains f at one point of the context space and nowhere else, and must be re-measured when the context changes. Scale alone would be an argument about sample size; persistence is an argument about what a bit buys.

Consequence for the model. An entity encoder is therefore fit where its data is, and the contingent corpus is spent only on what nothing else can pay for. This is the cut at H of Fig. 1c: F_θ maps persistent entities into the shared representation, and the phenotype labels fit only $\mathcal{T}_\psi$ and $\mathcal{R}_\phi$. The capacity arithmetic is one-sided. A sequence encoder carries 10^8–10^{10} parameters, whereas the operator and decoder together carry $\sim 5 \times 10^4$; and the *usable* supervision in the phenotype corpus is far below its compressed size, because the measurements are noisy and strongly redundant – interaction matrices are approximately low rank – which places the effective supervision nearer 10^6–10^7 bits than 10^{10}. Fitting an entity encoder from phenotype labels is thus underdetermined by orders of magnitude, while fitting the operator and decoder is not. Supplementary Note 6 is the empirical counterpart: across ~ 17 gene encodings, no pretrained biological embedding beats a one-hot code on fitness, which is what one expects when the labels cannot pay for a representation.

An objection, and what persistence answers. It may be objected that only the *yeast* entities are needed, and that the yeast proteome – some 3×10^6 residues, on the order of 10^7 bits – is smaller than the phenotype corpus, so the accounting would favour the labels. The objection is correct about the arithmetic and inverts the argument. An entity encoder is not fit on the yeast slice; it is fit on the whole archive and *applied* to the yeast slice, because a protein sequence means the same thing in every organism. Fungal DNA language models are built in exactly this way, pretrained across many fungal genomes and then read out on one [15]. That transfer is precisely what persistence buys, and it is why the relevant comparison is the full archive against the phenotype corpus rather than one organism's parts list against it.

Precision. Four things are not claimed. First, no number here is an information content: by Proposition 5, L_C bounds Kolmogorov complexity from above and is a codelength under `gzip`'s model, so only the ratio, and only to order of magnitude, is asserted. Second, compressed storage is not usable supervision; both sides are upper bounds on their effective content, the phenotype side discounted by noise and low rank, the entity side by homology redundancy. The latter is worth stating plainly, because it is the mechanism the architecture exploits: a protein language model compresses TrEMBL far better than `gzip` does, and the gap between the two is exactly the evolutionary structure a pretrained encoder supplies and `gzip` cannot see. Third, the archives overlap – RefSeq transcripts also occur in

`nt` – so Supplementary Table 2 reports a conservative total that omits the duplication. Fourth, corpora of *predicted* structures are excluded, being model outputs rather than observations; including them would widen the separation while weakening its interpretation.

Supplementary Note 6: a classical-machine-learning case study on gene representation and pooling ✗

Before introducing the Cell Graph Transformer we ask what simple models already achieve, and where they stall – because the answer dictates two architectural choices. Trained on 10^3–10^5 instances over ∼17 gene encodings, classical models (Supplementary Table 3, Supplementary Fig. 4) show a sharp split: *knockout fitness is essentially saturated by gene identity alone* – one-hot encoding reaches Pearson ≈ 0.87–0.90 at 10^5 and no pretrained biological embedding beats it – whereas *gene interactions are not learnable from any encoding*, plateauing at Pearson ≈ 0.4 (Spearman ≈ 0.47) regardless of representation or dataset size. The first result says a deep model is unnecessary for fitness; the second says the barrier to interactions lies not in the features but in how genes are combined.

Setup. A pooled bag of per-gene embeddings is about the simplest representation of a genome one can write down, and it is the object tested here. Each strain is featurized as $\phi = Mx$, where $x \in \{0,1\}^N$ ($N = 6607$) selects genes and the columns of M are per-gene vectors: one-hot ($M = I$), random (i.i.d., dimension 1–1000), or pretrained biological embeddings – the protein language models ESM2 and ProtT5, the codon language model CaLM, nucleotide-transformer windows, and a species-aware DNA language model [16]. The bag is taken over either the perturbed (deleted) genes or the intact genome and pooled by sum or mean, giving one fixed-length vector per strain (Supplementary Fig. 4a). Fitness data are single/double/triple deletion strains ($smf/dmf/tmf$, quality filter $\sigma_f < 0.15$); interaction data are an equal (≈50/50) mixture of digenic and trigenic Kuzmin 2018 measurements. Elastic net, random forest and support-vector regression are tuned per encoding by validation Spearman; reported values are held-out test scores, and the tables give the two nonlinear regressors, whose sweep is complete across all encodings and sizes.

Representation is about identity, not biology. On fitness a purely random code matches or exceeds every biological embedding once its dimension is large enough, and one-hot is best of all. The feature $\phi = Mx$ with ≤ 3 perturbed genes is a k-sparse recovery problem, so a random projection of dimension $d \gtrsim k \log N$ – a few hundred – already preserves gene identity (Johnson–Lindenstrauss; compressed sensing [cite: Johnson–Lindenstrauss 1984; Candès–Tao 2005]). Extra dimensions add no identity information, so accuracy rises with d and saturates at the exactly-orthogonal one-hot limit; a wider random vector cannot surpass it, and trails only for axis-aligned tree models, because a random projection is a rotation that removes the gene-aligned coordinates trees split on [cite: Grinsztajn et al. 2022]. Design consequence: preserve gene identity with a learnable token rather than hard-coding a biological prior at the reference cell.

The barrier is pooling, not features. Interactions are epistasis – defined by subtracting lower-order effects – so they live in the *combination* of perturbed genes. Additive sum/mean pooling of Mx is permutation-invariant and keeps only the bag of perturbed genes; the combination structure is discarded before the model sees it, and no choice of M can restore it. Hence the flat ≈ 0.47 ceiling across encodings and sizes. Within additive pooling, sum outperforms mean, and not by accident: sum is injective on the multiset of perturbed genes whereas mean discards its cardinality, making sum the strictly more expressive aggregator [cite: Xu et al. 2019, GIN]; concretely the number of deleted genes – and hence single/double/triple order – is linearly recoverable from a summed embedding but not a mean.

How far below the achievable does this plateau sit? The relevant bound is the *limit of experimental reproducibility* – the correlation between independent replicate measurements, which no predictor of a single measurement can exceed [cite: Ma et al. 2018, DCell]. Double-mutant fitness reproduces at r=0.89 across 211 duplicate genome-wide SGA screens [cite: Baryshnikova et al. 2010], so the one-hot fitness result (≈ 0.87–0.90) already sits *at* this ceiling – fitness is saturated by identity. Interaction scores are noisier but still highly reproducible: r=0.82–0.87 for digenic [cite: Baryshnikova et al. 2010][**?**] and r=0.74–0.81 for adjusted trigenic scores [**?**], a variance-weighted ceiling of r≈ 0.81–0.86 for the ≈50/50 mixture (dashed line, Supplementary Fig. 4). The classical plateau near 0.45 therefore lies far below the noise ceiling – the barrier is model capacity, not measurement error, and substantial reproducible interaction signal remains for a readout that preserves higher-order structure. Together these results motivate the two design choices of the Cell Graph Transformer – identity-preserving gene tokens (this note) and attention over the gene set in place of additive pooling (Supplementary Note 4) – so that higher-order structure survives to the readout.

Scope. These encodings are deliberately minimal. A pooled bag of pretrained gene vectors is among the simplest genome representations available, and the biological embeddings differ mainly in the domain they were pretrained on – protein, codon, or regulatory-DNA context – so what generality they carry is inherited from that pretraining rather than learned from phenotype here. Richer, learned cell representations exist and are advancing quickly [17]; the point is not that additive pooling of gene embeddings is the right way to represent a cell, but that even this minimal encoding already saturates fitness and stalls on interactions, which is what isolates the readout – not the features – as the object the Cell Graph Transformer must improve. This is an early-stage baseline, and it is meant to be one.

Supplementary Table 3 Classical-ML gene-representation benchmark – test Spearman ρ at $n = 10^3, 10^4, 10^5$ for random forest (RF) and support-vector regression (SVR), validation-selected best configuration per encoding. **Bold** = best encoding per column within each dataset; ± is the 5-fold CV s.d. ($n = 10^3, 10^4$; $n = 10^5$ single split). Regenerated by `traditional_ml-summary_table.py`.

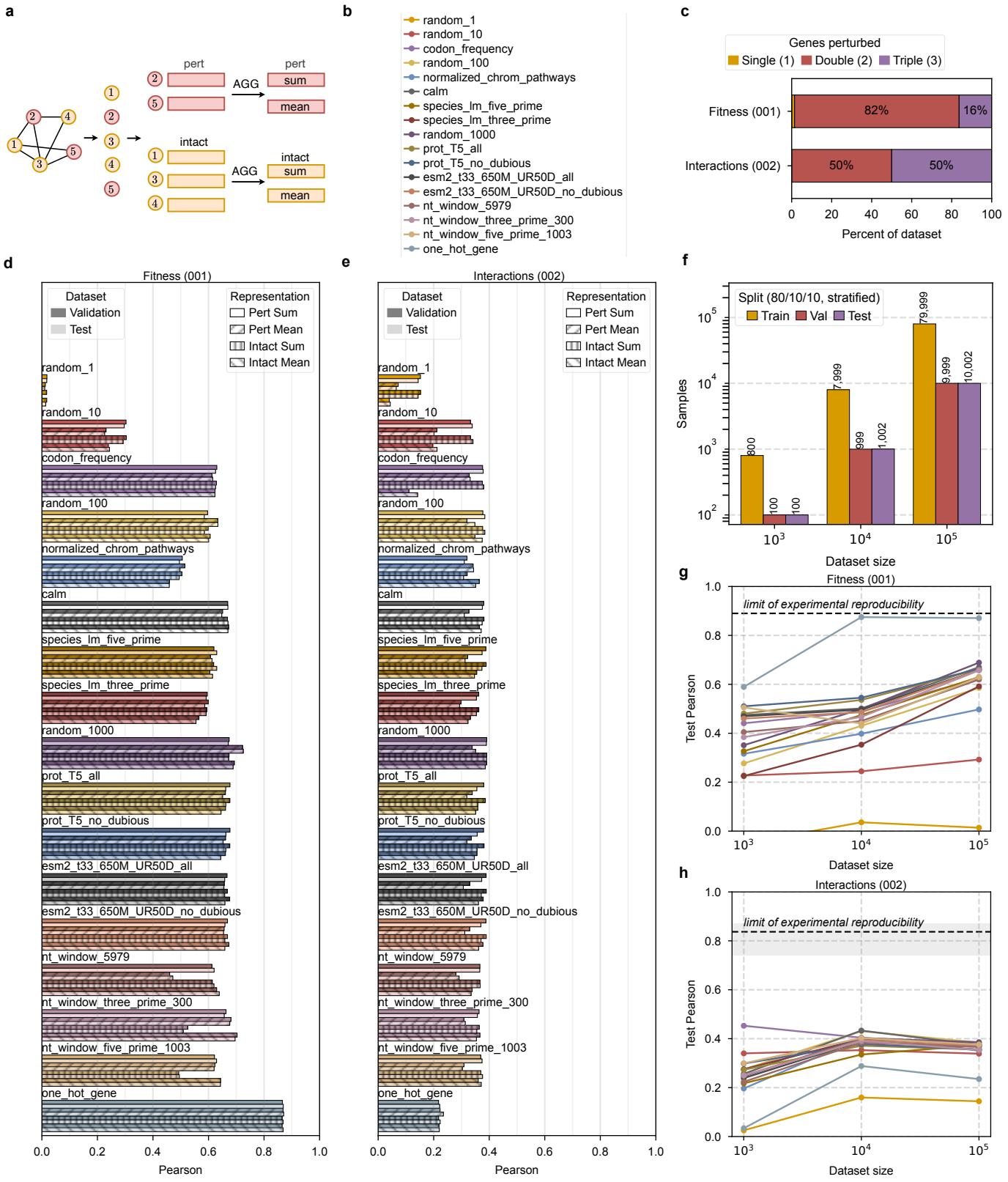
	RF			SVR		
Encoding	10^3	10^4	10^5	10^3	10^4	10^5
Fitness						
Random projection						
random (d=1)	0.013 ± 0.044	0.036 ± 0.015	0.033	-0.168 ± 0.036	0.044 ± 0.018	–
random (d=10)	0.138 ± 0.058	0.229 ± 0.023	0.263	0.075 ± 0.030	0.210 ± 0.016	–
random (d=100)	0.248 ± 0.057	0.422 ± 0.029	0.646	**0.532 ± 0.066**	0.642 ± 0.011	–
random (d=1000)	0.367 ± 0.053	0.565 ± 0.021	0.756	0.461 ± 0.046	0.763 ± 0.010	0.870 ± 0.002
Hand-crafted						
codon freq.	0.384 ± 0.055	0.452 ± 0.014	0.599	0.282 ± 0.043	0.286 ± 0.014	0.328
chrom. pathways	0.208 ± 0.053	0.353 ± 0.020	0.436	0.336 ± 0.060	0.307 ± 0.020	0.353
Biological						
CaLM	0.435 ± 0.067	0.510 ± 0.016	0.662	0.400 ± 0.054	0.719 ± 0.016	0.825
species LM up	0.296 ± 0.114	0.453 ± 0.027	0.635	0.356 ± 0.036	0.687 ± 0.012	0.818
species LM down	0.232 ± 0.110	0.391 ± 0.008	0.605	0.427 ± 0.030	0.651 ± 0.013	0.804
ProtT5	0.414 ± 0.050	0.522 ± 0.032	0.675	0.453 ± 0.080	0.737 ± 0.010	0.840
ProtT5 (nd)	0.421 ± 0.052	0.520 ± 0.030	0.677	0.453 ± 0.080	0.738 ± 0.010	0.841
ESM2	0.427 ± 0.049	0.510 ± 0.017	0.669	0.479 ± 0.043	0.774 ± 0.018	0.795
ESM2 (nd)	0.399 ± 0.061	0.506 ± 0.014	0.669	0.384 ± 0.031	0.777 ± 0.018	0.796
NT 5979	0.330 ± 0.038	0.427 ± 0.031	0.623	0.300 ± 0.043	0.802 ± 0.015	0.749
NT 3′	0.367 ± 0.066	0.477 ± 0.024	0.716	0.400 ± 0.061	0.797 ± 0.007	0.792
NT 5′	0.348 ± 0.061	0.432 ± 0.033	0.650	0.358 ± 0.051	0.788 ± 0.016	0.753
Identity						
one-hot	**0.561 ± 0.064**	**0.872 ± 0.002**	**0.886**	0.445 ± 0.060	**0.803 ± 0.009**	**0.900**
Interaction						
Random projection						
random (d=1)	0.183 ± 0.045	0.218 ± 0.022	0.209	0.240 ± 0.090	0.226 ± 0.017	0.209
random (d=10)	0.366 ± 0.120	0.461 ± 0.020	0.422	0.378 ± 0.100	0.470 ± 0.019	0.422
random (d=100)	0.411 ± 0.067	0.492 ± 0.013	0.451	0.415 ± 0.063	0.472 ± 0.022	0.445
random (d=1000)	0.322 ± 0.075	0.498 ± 0.015	0.466	**0.451 ± 0.088**	0.458 ± 0.383	**0.467**
Hand-crafted						
codon freq.	0.432 ± 0.089	0.487 ± 0.015	0.459	0.381 ± 0.078	0.486 ± 0.016	0.443
chrom. pathways	0.301 ± 0.092	0.487 ± 0.019	0.450	0.267 ± 0.080	0.446 ± 0.025	0.431
Biological						
CaLM	0.327 ± 0.081	**0.514 ± 0.021**	0.461	0.359 ± 0.096	0.484 ± 0.019	0.461
species LM up	0.404 ± 0.057	0.485 ± 0.009	**0.468**	0.448 ± 0.081	0.492 ± 0.016	0.461
species LM down	0.388 ± 0.064	0.485 ± 0.014	0.460	0.401 ± 0.067	0.477 ± 0.017	0.452
ProtT5	0.376 ± 0.065	0.485 ± 0.021	0.466	0.332 ± 0.103	0.493 ± 0.012	0.455
ProtT5 (nd)	0.353 ± 0.064	0.478 ± 0.021	0.468	0.332 ± 0.103	0.493 ± 0.012	0.455
ESM2	0.387 ± 0.062	0.501 ± 0.019	0.461	0.395 ± 0.083	**0.506 ± 0.017**	0.466
ESM2 (nd)	0.369 ± 0.061	0.490 ± 0.016	0.464	0.395 ± 0.083	**0.506 ± 0.017**	0.466
NT 5979	**0.435 ± 0.090**	0.453 ± 0.014	0.462	0.377 ± 0.081	0.491 ± 0.014	0.461
NT 3′	0.408 ± 0.124	0.500 ± 0.017	0.461	0.359 ± 0.078	0.485 ± 0.007	0.454
NT 5′	0.329 ± 0.070	0.502 ± 0.013	0.467	0.375 ± 0.085	0.461 ± 0.012	0.462
Identity						
one-hot	0.193 ± 0.051	0.401 ± 0.022	0.446	0.294 ± 0.078	0.478 ± 0.018	0.464

Supplementary Table 4 Classical-ML gene-representation benchmark – test MSE ($\times 10^{-3}$) at $n = 10^3, 10^4, 10^5$ for random forest (RF) and support-vector regression (SVR), validation-selected best configuration per encoding. **Bold** = best encoding per column within each dataset; ± is the 5-fold CV s.d. ($n = 10^3, 10^4$; $n = 10^5$ single split). †SVR fit diverged: configurations are selected by validation Spearman, which is scale-invariant, so a well-ranked fit can still be badly scale-calibrated and carry an enormous squared error. Regenerated by `traditional_ml-summary_table.py`.

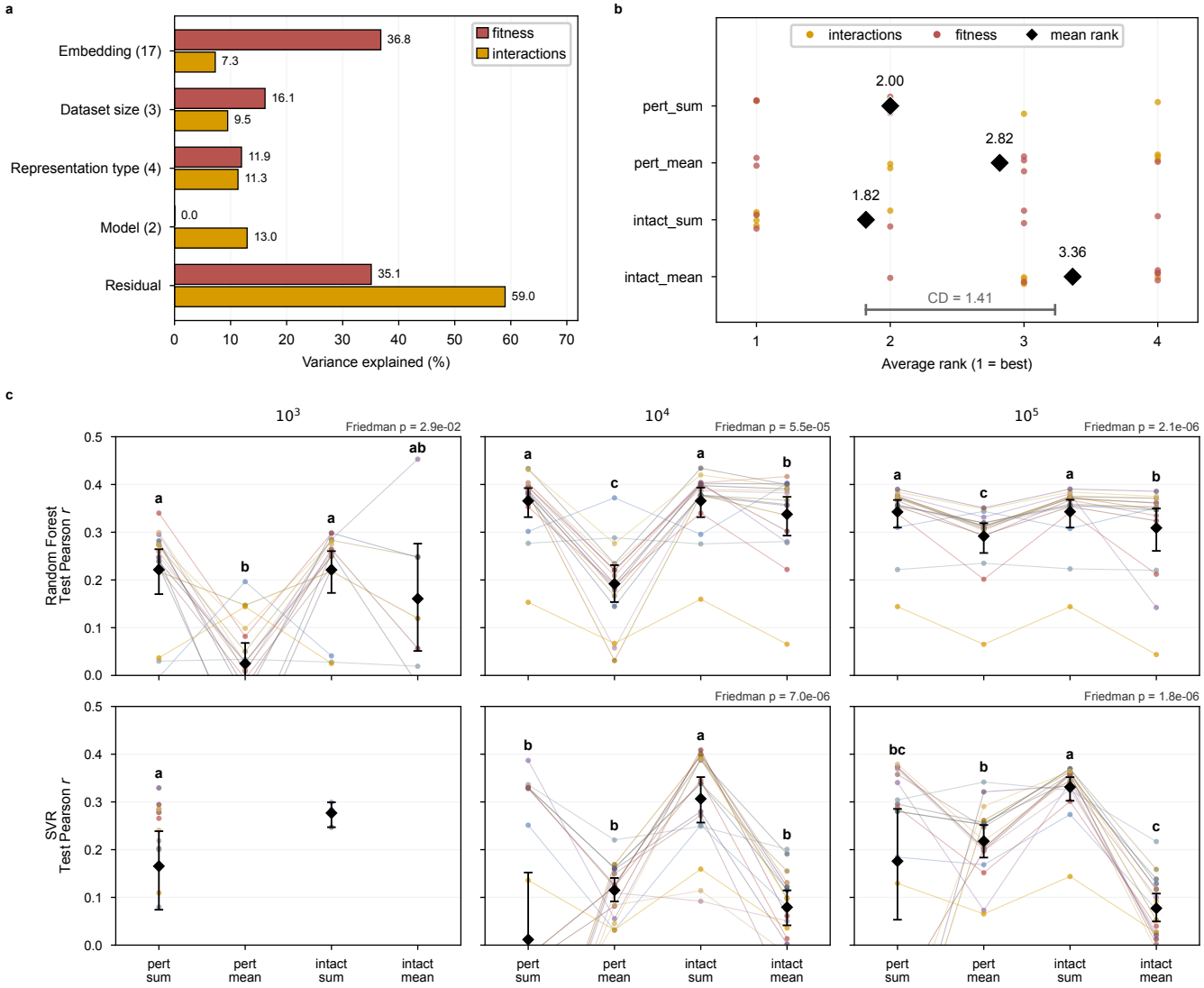
	RF			**SVR**		
Encoding	10^3	10^4	10^5	10^3	10^4	10^5
Fitness						
Random projection						
random (d=1)	25.6 ± 3.0	19.9 ± 0.7	21.3	23.5 ± 3.0	19.9 ± 0.6	–
random (d=10)	22.3 ± 2.7	18.9 ± 0.8	19.4	23.1 ± 2.8	18.9 ± 0.8	–
random (d=100)	22.1 ± 2.6	16.6 ± 0.6	15.3	20.8 ± 2.7	11.8 ± 0.4	–
random (d=1000)	20.2 ± 2.3	15.8 ± 0.7	13.6	22.8 ± 2.6	8.2 ± 0.2	5.1 ± 0.2
Hand-crafted						
codon freq.	18.6 ± 2.5	15.1 ± 0.4	13.5	20.5 ± 2.8	18.1 ± 0.8	18.9
chrom. pathways	22.0 ± 2.8	16.9 ± 0.6	16.0	20.2 ± 2.6	17.8 ± 0.8	17.8
Biological						
CaLM	18.6 ± 2.8	14.9 ± 0.4	12.7	21.1 ± 2.9	9.4 ± 0.2	6.5
species LM up	21.0 ± 1.2	15.8 ± 0.8	13.7	20.1 ± 2.6	11.0 ± 0.7	6.8
species LM down	22.2 ± 1.8	17.4 ± 0.7	14.9	21.5 ± 3.5	10.9 ± 0.5	7.3
ProtT5	19.0 ± 2.4	14.5 ± 0.5	12.8	17.8 ± 2.4	8.0 ± 0.5	6.5
ProtT5 (nd)	18.8 ± 2.3	14.6 ± 0.4	12.8	17.8 ± 2.4	8.0 ± 0.5	6.3
ESM2	18.7 ± 2.4	15.4 ± 0.5	13.1	$\mathbf{17.7 \pm 2.6}$	7.3 ± 0.8	7.3
ESM2 (nd)	19.2 ± 2.6	15.4 ± 0.6	12.9	17.7 ± 2.8	7.2 ± 0.8	7.3
NT 5979	20.0 ± 2.3	16.3 ± 0.6	14.3	28.7 ± 2.1	5.9 ± 0.4	8.7
NT 3′	21.1 ± 2.7	15.9 ± 0.6	13.5	19.7 ± 2.8	6.1 ± 0.4	9.9
NT 5′	20.4 ± 2.3	16.3 ± 0.8	14.0	20.8 ± 2.8	6.6 ± 0.4	10.0
Identity						
one-hot	$\mathbf{17.2 \pm 4.3}$	$\mathbf{4.8 \pm 0.3}$	**5.4**	20.2 ± 2.9	$\mathbf{5.9 \pm 0.4}$	**4.5**
Interaction						
Random projection						
random (d=1)	4.4 ± 0.9	3.6 ± 0.4	3.8	4.3 ± 0.9	3.9 ± 0.3	4.6×10^{16} †
random (d=10)	$\mathbf{3.9 \pm 1.1}$	3.2 ± 0.4	3.4	4.3 ± 0.9	$\mathbf{3.7 \pm 0.3}$	4.6
random (d=100)	4.1 ± 1.1	$\mathbf{2.9 \pm 0.4}$	3.3	$\mathbf{4.1 \pm 0.9}$	4.9 ± 0.3	4.4
random (d=1000)	4.2 ± 1.2	3.1 ± 0.4	**3.3**	4.4 ± 0.9	5.7×10^{13} †	4.3
Hand-crafted						
codon freq.	3.9 ± 1.1	3.0 ± 0.4	3.3	4.1 ± 1.0	4.2 ± 0.4	4.8
chrom. pathways	4.3 ± 1.0	3.1 ± 0.4	3.4	4.4 ± 0.9	4.1 ± 0.4	4.8
Biological						
CaLM	4.4 ± 1.1	2.9 ± 0.4	3.3	4.2 ± 1.1	4.2 ± 0.3	4.6
species LM up	4.2 ± 1.1	3.1 ± 0.4	3.4	5.3 ± 1.2	3.9 ± 0.4	4.6
species LM down	4.2 ± 1.0	3.1 ± 0.4	3.4	4.9 ± 1.2	4.4 ± 0.4	4.2
ProtT5	4.2 ± 1.1	3.1 ± 0.4	3.4	4.3 ± 1.0	4.0 ± 0.4	4.5
ProtT5 (nd)	4.3 ± 1.1	3.1 ± 0.4	3.4	4.3 ± 1.0	4.0 ± 0.4	4.5
ESM2	4.1 ± 1.1	3.1 ± 0.4	3.4	4.6 ± 1.2	4.2 ± 0.4	4.2
ESM2 (nd)	4.3 ± 1.1	3.1 ± 0.4	3.4	4.6 ± 1.2	4.2 ± 0.4	4.2
NT 5979	4.3 ± 1.0	3.2 ± 0.4	3.4	4.7 ± 1.2	4.3 ± 0.4	**4.2**
NT 3′	4.1 ± 1.2	3.1 ± 0.4	3.4	5.9 ± 0.9	4.7 ± 0.3	4.6
NT 5′	4.3 ± 0.9	3.0 ± 0.4	3.4	4.2 ± 1.1	4.5 ± 0.4	4.6
Identity						
one-hot	4.7 ± 1.1	3.5 ± 0.4	3.8	4.5 ± 0.9	3.8 ± 0.4	4.2

Supplementary Table 5 Classical-ML gene-representation benchmark – test Pearson r at $n = 10^3, 10^4, 10^5$ for random forest (RF) and support-vector regression (SVR), validation-selected best configuration per encoding. **Bold** = best encoding per column within each dataset; ± is the 5-fold CV s.d. ($n = 10^3, 10^4$; $n = 10^5$ single split). Regenerated by `traditional_ml-summary_table.py`.

	RF			SVR		
Encoding	10^3	10^4	10^5	10^3	10^4	10^5
Fitness						
Random projection						
random (d=1)	-0.060 ± 0.056	0.069 ± 0.023	0.014	-0.027 ± 0.019	0.066 ± 0.025	–
random (d=10)	0.227 ± 0.079	0.244 ± 0.031	0.296	0.136 ± 0.046	0.235 ± 0.030	–
random (d=100)	0.236 ± 0.034	0.430 ± 0.024	0.601	0.506 ± 0.049	0.655 ± 0.009	–
random (d=1000)	0.446 ± 0.100	0.502 ± 0.022	0.689	0.448 ± 0.039	0.791 ± 0.007	0.881 ± 0.004
Hand-crafted						
codon freq.	0.490 ± 0.038	0.512 ± 0.009	0.625	0.371 ± 0.061	0.311 ± 0.022	0.341
chrom. pathways	0.250 ± 0.066	0.394 ± 0.017	0.497	0.392 ± 0.068	0.336 ± 0.025	0.404
Biological						
CaLM	0.493 ± 0.051	0.529 ± 0.007	0.670	0.393 ± 0.087	0.738 ± 0.015	0.837
species LM up	0.317 ± 0.103	0.469 ± 0.034	0.630	0.389 ± 0.036	0.709 ± 0.022	0.829
species LM down	0.225 ± 0.091	0.368 ± 0.021	0.592	0.410 ± 0.038	0.696 ± 0.026	0.815
ProtT5	0.480 ± 0.068	0.549 ± 0.022	0.662	0.502 ± 0.045	0.777 ± 0.020	0.836
ProtT5 (nd)	0.510 ± 0.074	0.545 ± 0.021	0.663	0.502 ± 0.045	0.776 ± 0.019	0.842
ESM2	0.480 ± 0.062	0.496 ± 0.010	0.659	**0.516 ± 0.055**	0.803 ± 0.024	0.815
ESM2 (nd)	0.458 ± 0.069	0.496 ± 0.010	0.659	0.501 ± 0.071	0.805 ± 0.025	0.815
NT 5979	0.406 ± 0.073	0.444 ± 0.027	0.639	0.230 ± 0.076	0.842 ± 0.015	0.781
NT 3′	0.328 ± 0.069	0.482 ± 0.017	0.696	0.416 ± 0.084	0.837 ± 0.014	0.801
NT 5′	0.389 ± 0.064	0.441 ± 0.030	0.643	0.340 ± 0.061	0.826 ± 0.017	0.773
Identity						
one-hot	**0.589 ± 0.090**	**0.873 ± 0.010**	**0.871**	0.491 ± 0.055	**0.844 ± 0.013**	**0.902**
Interaction						
Random projection						
random (d=1)	0.073 ± 0.068	0.158 ± 0.016	0.145	0.110 ± 0.085	0.158 ± 0.014	0.144
random (d=10)	**0.337 ± 0.091**	0.353 ± 0.036	0.339	0.279 ± 0.087	0.339 ± 0.033	0.329
random (d=100)	0.280 ± 0.059	**0.448 ± 0.039**	0.373	0.284 ± 0.095	0.383 ± 0.037	0.361
random (d=1000)	0.213 ± 0.106	0.401 ± 0.033	**0.390**	0.155 ± 0.037	0.388 ± 0.298	**0.384**
Hand-crafted						
codon freq.	0.326 ± 0.100	0.414 ± 0.035	0.377	0.292 ± 0.100	0.398 ± 0.038	0.346
chrom. pathways	0.139 ± 0.063	0.395 ± 0.038	0.350	0.097 ± 0.065	0.363 ± 0.044	0.345
Biological						
CaLM	0.190 ± 0.069	0.441 ± 0.031	0.379	0.247 ± 0.084	0.381 ± 0.033	0.364
species LM up	0.239 ± 0.073	0.381 ± 0.038	0.372	**0.302 ± 0.099**	0.400 ± 0.029	0.362
species LM down	0.260 ± 0.051	0.400 ± 0.039	0.342	0.273 ± 0.093	0.398 ± 0.036	0.373
ProtT5	0.276 ± 0.068	0.381 ± 0.027	0.352	0.223 ± 0.094	**0.410 ± 0.034**	0.372
ProtT5 (nd)	0.252 ± 0.073	0.384 ± 0.028	0.354	0.223 ± 0.094	**0.410 ± 0.034**	0.372
ESM2	0.280 ± 0.083	0.389 ± 0.037	0.368	0.270 ± 0.085	0.405 ± 0.037	0.382
ESM2 (nd)	0.243 ± 0.085	0.388 ± 0.038	0.372	0.270 ± 0.085	0.405 ± 0.037	0.382
NT 5979	0.224 ± 0.108	0.376 ± 0.023	0.360	0.300 ± 0.097	0.402 ± 0.039	0.380
NT 3′	0.268 ± 0.109	0.395 ± 0.035	0.358	0.186 ± 0.067	0.404 ± 0.028	0.360
NT 5′	0.247 ± 0.046	0.430 ± 0.039	0.368	0.241 ± 0.088	0.377 ± 0.029	0.369
Identity						
one-hot	0.131 ± 0.073	0.288 ± 0.040	0.343	0.118 ± 0.052	0.399 ± 0.029	0.373



Supplementary Figure 4 Classical machine-learning baselines: fitness is saturable, interactions are not. (a) Featurization. Each strain is reduced to a single fixed-length vector by mapping its genes through a per-gene encoder and pooling: the bag is taken over either the perturbed (deleted) genes or the intact genome and aggregated by sum or mean. (b) The seventeen gene representations, grouped as identity (one-hot), random projections (d=1–1000), hand-crafted (codon frequency, chromosome/pathway features), and pretrained biological encoders (CaLM, a species-aware DNA language model [16], nucleotide-transformer windows, ProtT5, ESM2). (c) Perturbation composition of the two datasets: knockout fitness (001; single/double/triple deletions) and the digenic/trigenic interaction mixture (002). (d,e) Validation and test Pearson r at n=10^5 for random forest across all representations and the four pooling variants (perturbed/intact × sum/mean), hyperparameters selected on validation MSE; Spearman and MSE views, and the support-vector and elastic-net baselines, are tabulated in Supplementary Table 3–5. (f) Stratified 80/10/10 train/validation/test splits at each dataset size. (g,h) Best test Pearson per representation as dataset size grows n=10^3–10^5; the dashed line and shaded band mark the limit of experimental reproducibility (fitness r≈0.89; interactions r≈0.81–0.86). Fitness is saturated by gene identity – one-hot reaches Pearson ≈ 0.87 at n=10^5, approaching the reproducibility limit, and no pretrained embedding surpasses it – whereas interactions plateau near Pearson ≈ 0.4 regardless of representation or scale, the empirical backing for the information-accounting cut of Supplementary Note 5.



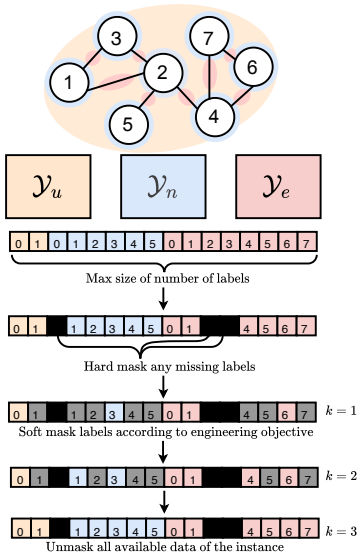
Supplementary Figure 5 Representation type – perturbed vs intact gene bag, sum vs mean pooling – is a consistent, second-order design choice. Every record is the best run by validation R^2 for one (task, model, dataset size, embedding, representation type) combination, scored as held-out test Pearson r (the same selection rule as Supplementary Fig. 4). (a) Variance decomposition, computed separately for fitness (n=384 records) and gene interactions (n=350): type-II ANOVA over main effects, reporting $\eta^2 = \mathrm{SS}_{\mathrm{factor}}/\mathrm{SS}_{\mathrm{total}}$; parenthetical counts are each factor's number of levels, and the residual absorbs factor interactions and run-to-run noise. For interactions, representation type (11.3%) explains more variance than embedding identity (7.3%), whereas the large fitness embedding share (36.8%) restates the one-hot saturation of Supplementary Fig. 4. (b) Average rank of the four representation types across the 11 complete task×model×scale configurations (dot = one configuration's rank, colored by task; diamond = mean rank; 1 = best). Friedman omnibus p=0.016; the ruler is the Nemenyi critical difference (CD=1.41, α=0.05), which formally separates only the extremes, intact_sum (mean rank 1.82) vs intact_mean (3.36). (c) Gene-interaction detail: test Pearson by representation type for random forest and SVR at each dataset size; faint line = one embedding (17 total), diamond = mean with 95% bootstrap CI. Letters are a compact letter display – types sharing a letter are not significantly different (pairwise Wilcoxon, BH-FDR corrected), shown only when that panel's Friedman omnibus is significant (p above each panel). intact_sum sits in the top letter group in every complete panel and pert_mean is significantly worse beyond n=10^3; the missing points at SVR 10^3 reflect an incomplete sweep.

Supplementary Table 6 Phenotype datasets integrated into the TorchCell knowledge graph. Genotypes are labeled instances; Envs. is the number of distinct media/condition contexts; Label type is the graph object that carries the label (*global* whole-cell scalar, *node* gene-level, *edge* digenic, *hyperedge* trigenic). smf/dmf/tmf: single/double/triple-mutant fitness; dmi/tmi: di-/trigenic interaction.

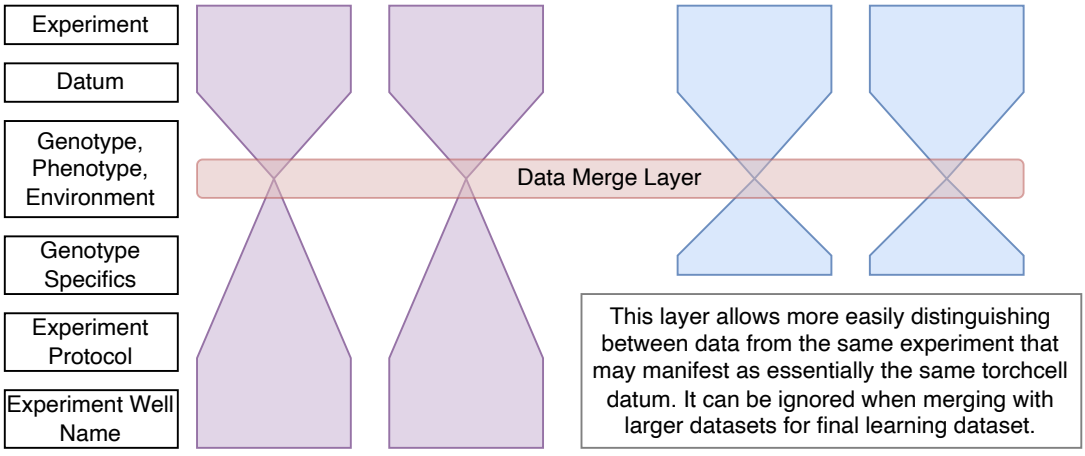
Dataset	Genotypes	Envs.	Phenotype	Label type	Description	Status
Baryshnikova_2010	6,022	1	smf	global	growth rate	✓
Costanzo_2016_smf	21,718	2	smf	global	growth rate	✓
Costanzo_2016_dmf	20,705,612	2	dmf	global	growth rate	✓
Costanzo_2016_dmi	20,705,612	2	dmi	edge	gene interaction	✓
Kuzmin_2018_smf	1,539	1	smf	global	growth rate	✓
Kuzmin_2018_dmf	410,399	1	dmf	global	growth rate	✓
Kuzmin_2018_tmf	91,111	1	tmf	global	growth rate	✓
Kuzmin_2018_dmi	410,399	1	dmi	edge	gene interaction	✓
Kuzmin_2018_tmi	91,111	1	tmi	hyperedge	gene interaction	✓
Kuzmin_2020_smf	480	1	smf	global	growth rate	✓
Kuzmin_2020_dmf	256,862	1	dmf	global	growth rate	✓
Kuzmin_2020_tmf	537,911	1	tmf	global	growth rate	✓
Kuzmin_2020_dmi	256,862	1	dmi	edge	gene interaction	✓
Kuzmin_2020_tmi	537,911	1	tmi	hyperedge	gene interaction	✓
SGD_essential	1,101	1	viable	global	viability	✓
SynthLethalYeast	1,400	–	viable	global	viability	✓
SynthRescueYeast	6,948	–	viable	global	viability	✓
scmd2_2005	4,718	1	morphology	global	cell morphology	✓
scmd2_2018	1,112	1	morphology	global	cell morphology	✓
scmd2_2022	1,982	1	morphology	global	cell morphology	✓
ODuibhir_2014	1,312	1	expr	global, node	sm microarray expression	✓
Kemmeren_2014	1,484	1	expr	global, node	sm microarray expression	✓
Sameith_2015	72	1	expr	global, node	dm microarray expression	✓
Zelezniak_2018	97	1	prot., met.	global, node	sm kinase deletion mutants	✓
Wildenhain_2015	195	4,915	smf	global	drug tolerance	in prog.
Lian_2017	18,000	1	AID	global	furfural tolerance	in prog.
FitDb	∼6,000	1,144	smf	global	growth rate	in prog.

Supplementary Table 7 Interaction and regulatory graphs (databases) integrated into the TorchCell knowledge graph, with node and edge counts over the shared gene vocabulary. **Physical** and **Regulatory** are the curated graphs taken directly from SGD; the remaining rows add TFLink and the nine STRING v12 channels. Relative to the SGD-native baseline (Physical + Regulatory), aggregating these additional sources increases connectivity by 4.16× in nodes and 16.24× in edges – the justification for aggregating across sources rather than relying on a single database. (STRING v11, used in earlier experiments, is reported separately to avoid mixing graph versions.)

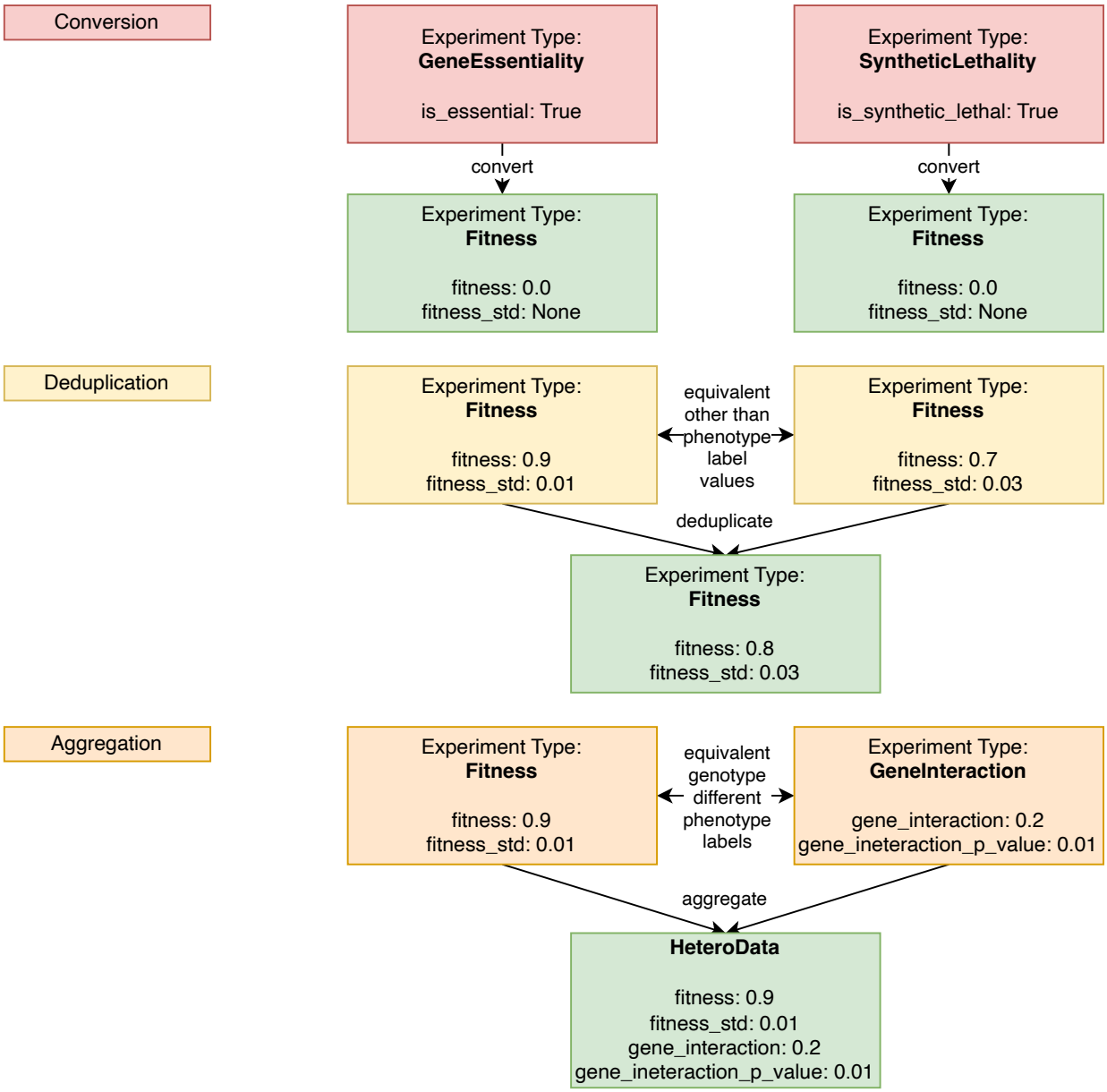
Graph	Nodes	Edges
Regulatory	3,632	9,753
Physical	5,721	139,463
TFLink	5,092	201,898
STRING12 Neighborhood	2,204	147,874
STRING12 Fusion	3,095	11,810
STRING12 Cooccurrence	2,615	11,115
STRING12 Coexpression	6,503	1,002,538
STRING12 Experimental	6,036	825,101
STRING12 Database	4,044	73,818
Sum (all graphs)	38,942	2,423,370
Sum (Physical, Regulatory)	9,353	149,216
Factor increase	4.16×	16.24×



Supplementary Figure 6 TorchCell: ontology-unified knowledge graph and the perturbation-operator cell representation. Panels (a)–(d).



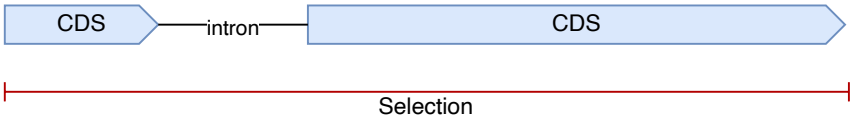
Supplementary Figure 7 Ontology data model (hourglass). Each experiment funnels genotype, phenotype, environment, and experiment-specific metadata through a shared Data Merge Layer that distinguishes records from the same experiment; the layer can be ignored when merging into larger datasets.



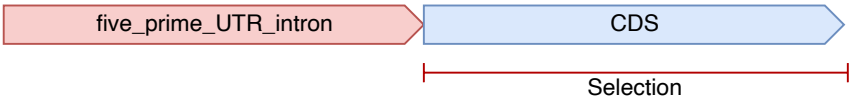
Supplementary Figure 8 Knowledge-graph normalization in three stages. Conversion maps phenotype experiment types (GeneEssentiality, SyntheticLethality) to a common Fitness type; deduplication collapses records equivalent other than their phenotype-label values; aggregation combines equivalent-genotype records with different phenotype labels (Fitness, GeneInteraction) into a single HeteroData instance.

DNA Selection
To comply with the standard format using in DNA language models we want to select the DNA sequence that has the highest probability of having the outermost start and stop codons for the CDS of the the gene. This means we will ignore five prime UTR introns if they are not internal to the gene. Below we outline how we handle intron cases:

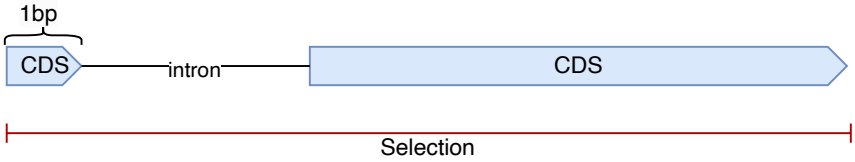
YFL039C - Internal Intron
Selectin - Entire gene



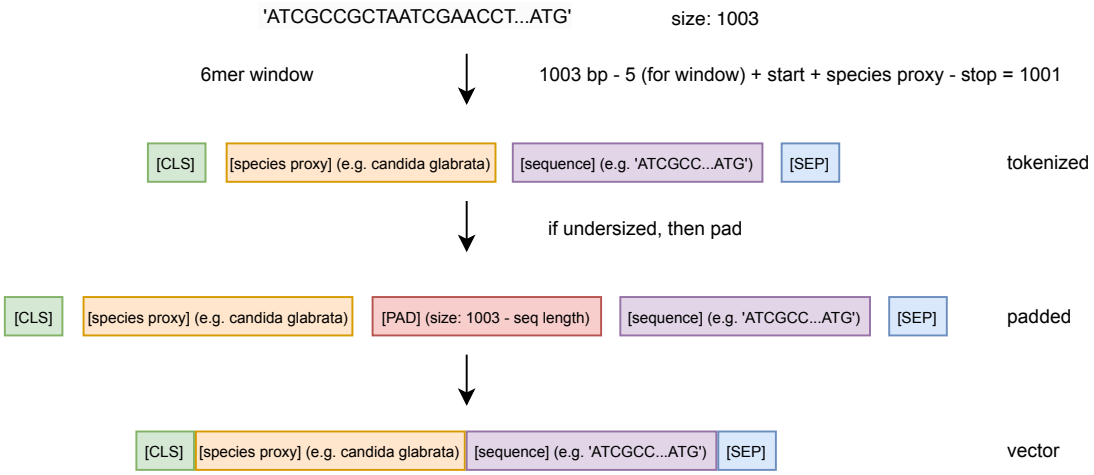
YBL072C - Upstream five prime UTR intron
Selection - Gene



YIL111W - 1bp CDS
Selection - Gene (The only case with no start codon)



Supplementary Figure 9 DNA sequence selection for the sequence model. The coding sequence is taken from the outermost start to stop codon, ignoring five-prime UTR introns unless internal to the gene. Cases: internal intron (YFL039C), 5' UTR intron (YBL072C), and a 1 bp CDS lacking a start codon (YIL111W).



Supplementary Figure 10 DNA tokenization. A 1003 bp window is encoded as [CLS], species proxy, 6-mer sequence, [SEP]; undersized sequences are padded with [PAD] to a fixed length before vectorization.

[Suppl. Fig. – expression & multitask diagnostics]

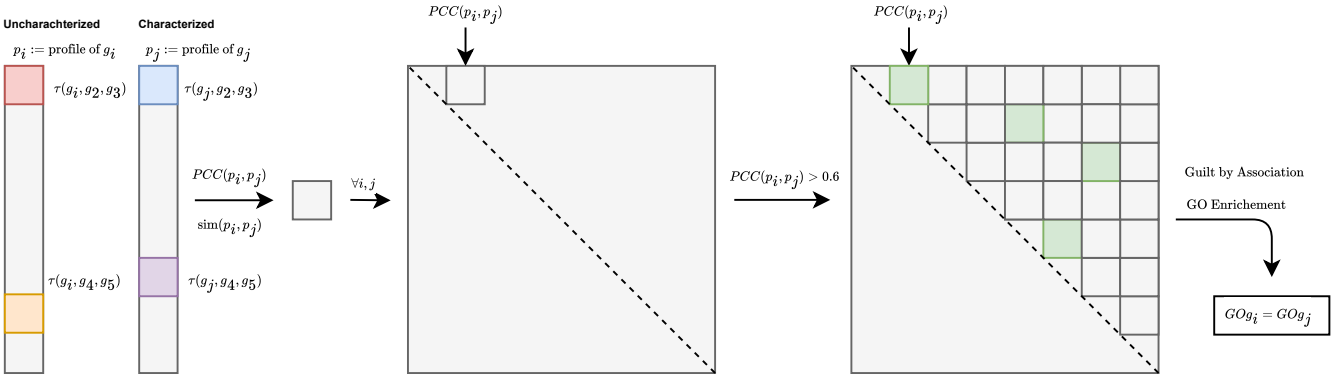
Knockout-transcriptome prediction diagnostics (per-gene calibration, top-gene predicted vs. actual) and cross-phenotype transfer from the shared latent embedding.

Supplementary Figure 11 Expression and multitask prediction diagnostics.

[Suppl. Fig. – inference at scale]

Large-scale inference pipeline over $\sim 10^6$ candidate perturbations (metabolic/kinase gene panels), with resource usage and the ranked strain-design output.

Supplementary Figure 12 Inference at scale for strain design.



Supplementary Figure 13 Guilt-by-association annotation of uncharacterized genes. (a)–(e) Triple-interaction profiles p are correlated (Pearson, PCC) across all gene pairs; pairs with PCC > 0.6 are linked, and GO enrichment over the resulting network transfers annotations ($\text{GO}_{g_i} = \text{GO}_{g_j}$). (f) Total quantification of triple interactions across the remaining yeast genome (coverage of uncharacterized genes). (g) The resulting new annotations transferred to previously uncharacterized genes.

References

[1] Stephanopoulos, G., Aristidou, A. A. & Nielsen, J. *Metabolic Engineering: Principles and Methodologies* (Elsevier, 1998).

[2] Bordbar, A., Monk, J. M., King, Z. A. & Palsson, B. O. Constraint-based models predict metabolic and associated cellular functions. *Nature Reviews Genetics* **15**, 107–120 (2014).

[3] Costanzo, M. *et al.* A global genetic interaction network maps a wiring diagram of cellular function. *Science* **353**, aaf1420–aaf1420 (2016).

[4] Kuzmin, E. *et al.* Systematic analysis of complex genetic interactions. *Science* **360**, eaao1729 (2018).

[5] Sapoval, N. *et al.* Current progress and open challenges for applying deep learning across the biosciences. *Nature Communications* **13**, 1728 (2022).

[6] Volk, M. J. *et al.* Metabolic Engineering: Methodologies and Applications. *Chemical Reviews* acs.chemrev.2c00403 (2022).

[7] Presnell, K. V. & Alper, H. S. Systems Metabolic Engineering Meets Machine Learning: A New Era for Data-Driven Metabolic Engineering. *Biotechnology Journal* **14**, 1800416 (2019).

[8] Culley, C., Vijayakumar, S., Zampieri, G. & Angione, C. A mechanism-aware and multiomic machine-learning pipeline characterizes yeast cell growth. *Proceedings of the National Academy of Sciences* **117**, 18869–18879 (2020).

[9] Cui, H. *et al.* scGPT: Toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods* **21**, 1470–1480 (2024).

[10] Kalfon, J., Samaran, J., Peyré, G. & Cantini, L. scPRINT: Pre-training on 50 million cells allows robust gene network predictions. *Nature Communications* **16**, 3607 (2025).

[11] Merzbacher, C., Mac Aodha, O. & Oyarzún, D. A. Accurate prediction of gene deletion phenotypes with Flux Cone Learning. *Nature Communications* **16**, 8492 (2025).

[12] Ahlmann-Eltze, C., Huber, W. & Anders, S. Deep-learning-based gene perturbation effect prediction does not yet outperform simple linear baselines. *Nature Methods* **22**, 1657–1661 (2025).

[13] Stewart, A. J. *et al.* TorchGeo: Deep learning with geospatial data. *Proceedings of the 30th International Conference on Advances in Geographic Information Systems* 1–12 (2022).

[14] Roohani, Y., Huang, K. & Leskovec, J. Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nature Biotechnology* **42**, 927–935 (2024). [ZOTERO-PENDING – manual entry, reconcile before submission].

[15] Chao, K.-H. *et al.* Predicting dynamic expression patterns in budding yeast with a fungal DNA language model (2025).

[16] Karollus, A. *et al.* Species-aware DNA language models capture regulatory elements and their evolution. *Genome Biology* **25**, 83 (2024). [ZOTERO-PENDING – manual entry, reconcile before submission].

[17] Rosen, Y. *et al.* Universal cell embedding provides a foundation model for cell biology. *Nature* (2026). [ZOTERO-PENDING – manual entry, reconcile before submission; Nature, online ahead of print (2026-07-08), no volume/pages assigned yet].

Draft word-budget report (body prose vs `\cb` budget; figures/captions excluded; auto-generated at build)

Section	Words	Budget	%
Abstract	140	150	93
Introductionintrotent	362	570	64
Resultsresultstodo	679	1900	36
A shared ontology and knowledge graph unify metabolic-engineering datar1todo	457	380	120
The Cell Graph Transformer predicts trigenic gene-gene interactions at state of the artr2todo	53	380	14
One embedding, many phenotypes: expression, morphology, and fitnessr3todo	20	380	5
Natural genetic variation versus model-designed perturbationsr4todo	21	380	6
Drug exposure and robustness of the learned cell representationr-robusttodo	42	380	11
Extending to metabolism and strain designr5todo	33	380	9
Discussiondiscussiontodo	170	570	30
Methodsmethodstodo	2472	—	—